

Laporan Tugas Besar 2 IF2211 - Strategi Algoritma
Pemanfaatan Algoritma BFS dan DFS dalam
Mekanisme Penelusuran CSS pada pohon Document
Object Model



Disusun oleh:

Stanislaus Ardy Bramantyo	18223057
Leonard Arif Sutiono	18223120
Muhammad Dzaky Atha Fadhillah	18223124

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

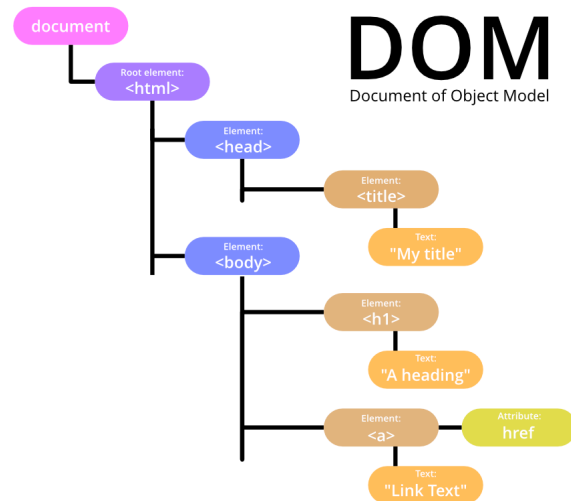
Daftar Isi

Daftar Isi.....	2
Bab 1.....	4
Deskripsi tugas.....	4
Bab 2.....	8
Landasan Teori.....	8
2.1. Algoritma Traversal Graf.....	8
2.2. Breadth First Search (BFS).....	9
2.3. Depth First Search (DFS).....	13
Bab 3.....	16
Aplikasi BFS dan DFS.....	16
3.1 Relevansi Algoritma Traversal pada Struktur DOM.....	16
3.2 Strategi BFS untuk Pencarian Elemen.....	16
3.2.1. Prinsip Dasar.....	16
3.2.2. Alur Kerja BFS Sekuensial.....	16
3.2.3. Alur Kerja BFS Konkuren.....	17
3.2.4 Karakteristik BFS.....	17
3.3 Strategi DFS untuk Pencarian Elemen.....	18
3.3.1 Prinsip Dasar.....	18
3.3.2 Alur Kerja DFS Sekuensial.....	18
3.3.3 Alur Kerja DFS Konkuren.....	18
3.3.4 Karakteristik DFS.....	19
3.4 Pendekatan Pencarian CSS Selector.....	19
3.4.1 Struktur SearchQuery.....	19
3.4.2 Evaluasi Query pada Setiap Node.....	20
3.4.3 Combinator yang Didukung.....	20
3.5 Pemanfaatan Multithreading.....	21
3.5.1 Motivasi.....	21
3.5.2 Strategi Pembagian Kerja.....	21
3.5.3 Menjaga Konsistensi Urutan.....	21
3.6 Pemanfaatan LCA Binary Lifting.....	21
3.6.1 Motivasi.....	21
3.6.2 Struktur Data.....	22
3.6.3 Proses Pembangunan Indeks LCA.....	22
3.6.4 Penggunaan dalam Evaluasi Selector.....	22
3.7 Perbandingan BFS dan DFS dalam Konteks DOM.....	22
3.7.1 Perbedaan Hasil Pencarian.....	22
3.7.2 Perbandingan Performa.....	23
3.7.3 Konsumsi Memori.....	23
Bab 4.....	24

Implementasi dan Pengujian.....	24
4.1 Implementasi.....	24
4.1.1. Breadth First Search.....	24
4.1.2. Depth First Search.....	31
4.1.3. DomNode.....	38
4.1.4. Tree.....	40
4.1.5. Parser.....	44
4.1.6. Selector.....	50
4.2. Pengujian.....	59
4.2.1. Selektor Dasar.....	59
4.2.2 Selektor Combinator.....	63
Bab 5.....	66
Kesimpulan dan Saran.....	66
5.1. Kesimpulan.....	66
5.2. Saran.....	66
Lampiran.....	67
Daftar Pustaka.....	69

Bab 1

Deskripsi tugas

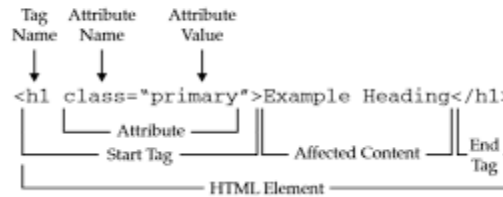


Gambar 1. Struktur Pohon DOM

<https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/>

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk *tree* yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram. Struktur sebuah file HTML pada umumnya meliputi elemen root `<html>` yang berisi dua bagian utama yaitu `<head>` dan `<body>`. Bagian `<head>` berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan `<body>` berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna. Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector. Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan. Komponen-komponen penting yang terdapat pada tugas besar ini antara lain:

1. Tags

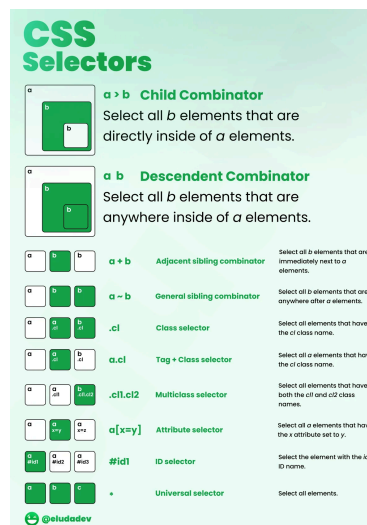


Gambar 2. Struktur Syntax HTML

<https://tutorial.techaltum.com/htmlTags.html>

Tag pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web. Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (< >). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring / sebelum nama tag. Terdapat beberapa jenis tag HTML yang *self-closing*. Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai node pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti parent, child, dan sibling. Struktur inilah yang memungkinkan dokumen HTML diproses sebagai sebuah pohon ketika dilakukan penelusuran menggunakan algoritma seperti BFS atau DFS.

2. CSS Selector



Gambar 3. Visualisasi CSS Selector

https://www.reddit.com/r/webdev/comments/ur6v5m/css_selectors_visually_explained/

CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Dengan selector, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak. Beberapa selector dasar yang umum digunakan adalah sebagai berikut.

- Tag Selector

Memilih elemen berdasarkan nama tag HTML. Contoh: p memilih semua elemen <p>.

- Class Selector

Memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (.). Contoh: .box

- ID Selector

Memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (#).

Contoh: #header

- Universal Selector

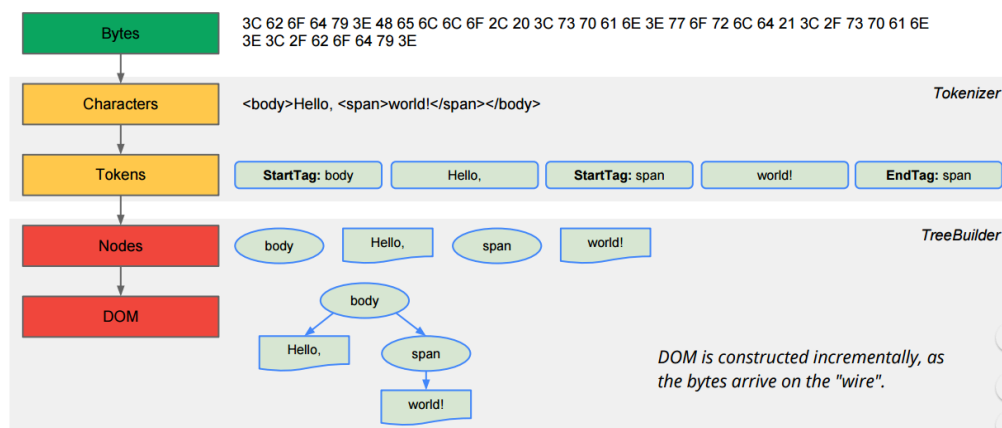
Memilih semua elemen HTML.

Contoh: *

Combinator merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri dari:

- Child Combinator (>)
- Descendent Combinator (" ")
- Adjacent Sibling Combinator (+)
- General Sibling Combinator (~)

3. HTML Parsing



Gambar 4. Proses Parsing Pohon HTML

<https://stackoverflow.com/questions/3150293/how-does-a-parser-for-example-html-work>

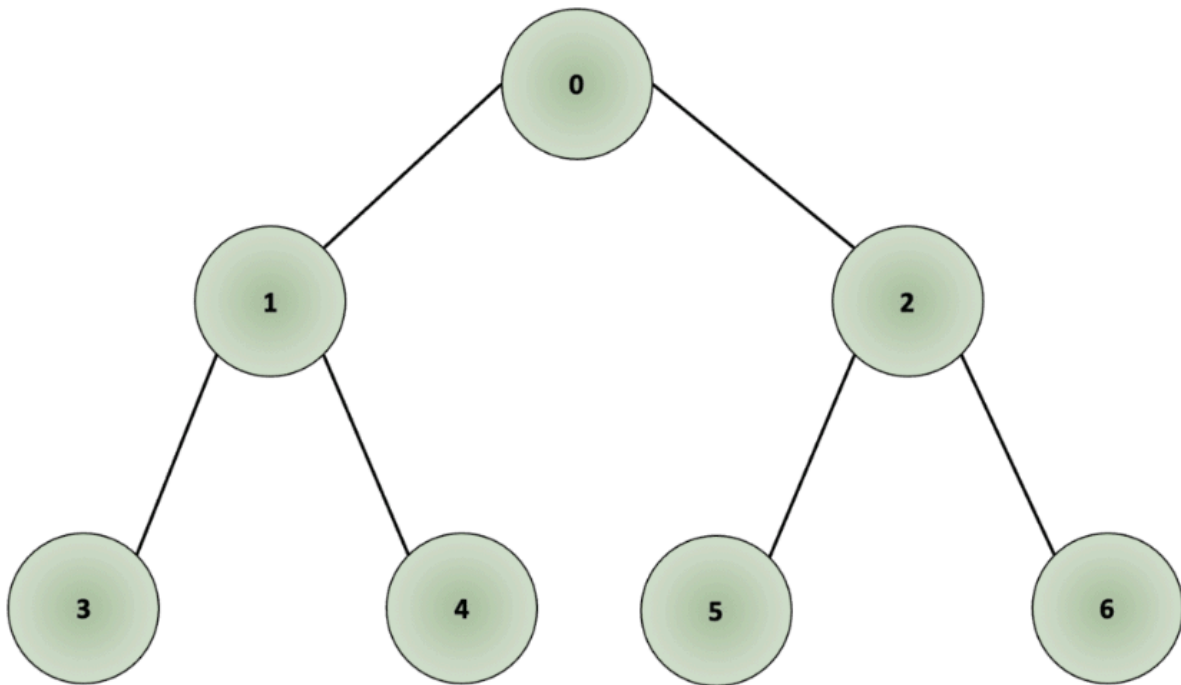
HTML Parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, parser memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag penutup, kemudian menyusunnya ke dalam struktur hierarki. Setiap tag HTML direpresentasikan sebagai node pada pohon. Tag yang berada di dalam tag lain akan menjadi child node, sedangkan tag yang membungkusnya menjadi parent node. Node paling atas pada struktur ini disebut root, yang umumnya merupakan elemen `<html>`. Dari node root tersebut, elemen-elemen lain seperti `<head>` dan `<body>` akan menjadi cabang yang membentuk struktur pohon DOM. Hasil dari proses parsing adalah DOM Tree, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma Breadth First Search (BFS) atau Depth First Search (DFS).

Bab 2

Landasan Teori

2.1. Algoritma Traversal Graf

Algoritma Traversal Graf adalah algoritma penelusuran graf yang mengunjungi setiap simpul yang terhubung di dalam suatu graf dengan cara yang sistematis. Pada algoritma ini, graf merupakan representasi sebuah persoalan sehingga melakukan traversal di dalam graf diartikan sebagai melakukan pencarian solusi persoalan yang direpresentasikan dengan graf tersebut. Secara visual, penelusuran graf dapat digambarkan sebagai pohon (*tree*) keputusan seperti berikut:



Gambar 2.1 Contoh simpul yang akan dilalui dalam sebuah graf yang divisualisasikan sebagai sebuah pohon keputusan

Saat menelusuri simpul-simpul dalam sebuah graf, terdapat beberapa permasalahan yang biasa dihadapi yaitu sebagai berikut:

- (1) Kasus dimana sebuah graf tidak terhubung sehingga tidak semua simpul dapat dijangkau

- (2) Dalam kasus *Cyclic Graph*, yaitu graf yang dimulai dan diakhiri pada simpul yang sama, harus dipastikan bahwa siklus tersebut tidak menyebabkan algoritma penelusuran masuk ke dalam perulangan tak terbatas
- (3) Algoritma mungkin perlu mengunjungi beberapa node lebih dari satu kali karena algoritma tidak menyimpan informasi apakah suatu node sudah pernah dikunjungi sebelum beralih ke node tersebut

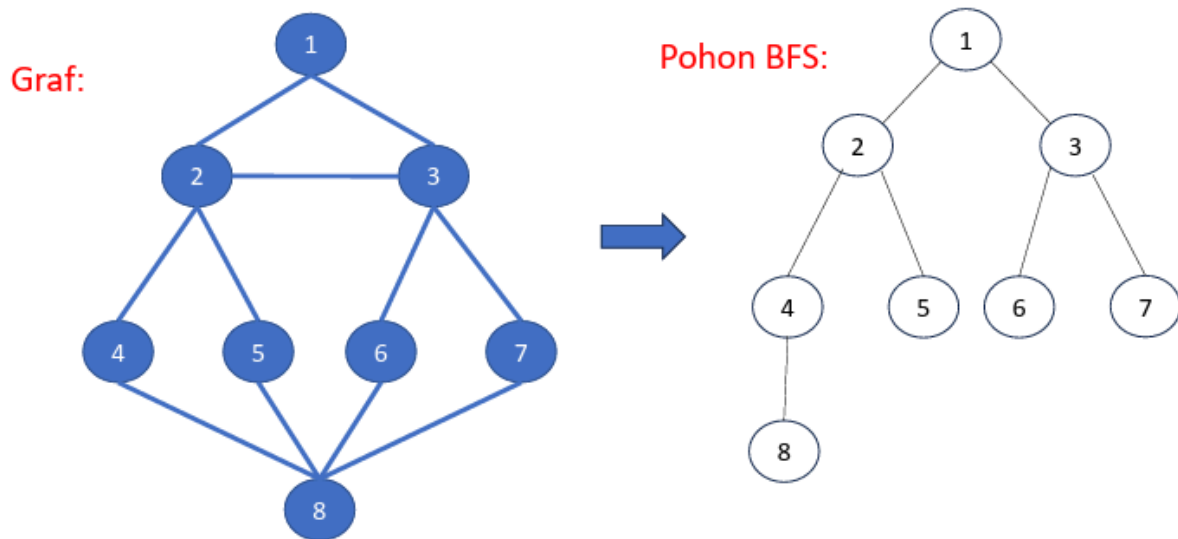
Algoritma penelusuran graf dapat menyelesaikan masalah kedua dan ketiga dengan menandai simpul sebagai telah dikunjungi apabila:

- (1) Pada awalnya tidak ada simpul yang ditandai sebagai telah dikunjungi
- (2) Ketika simpul dikunjungi, kita menandainya sebagai telah dikunjungi selama penelusuran
- (3) Simpul yang ditandai tidak dikunjungi untuk kedua kalinya untuk mencegah program memasuki perulangan penelusuran yang tidak berhenti

Terdapat 2 jenis algoritma pencarian solusi berbasis graf yang mana salah satunya yang diimplementasikan pada tugas besar ini adalah **Uninformed search**. Algoritma ini tidak memiliki informasi tambahan yang disediakan tentang seberapa dekat suatu simpul dengan tujuan (*goal*) sehingga satu-satunya informasi hanya berasal dari graf itu sendiri. Pada tugas besar ini, kami mengimplementasikan 2 algoritma penelusuran graf yang paling populer yaitu **Breadth First Search (BFS)** dan **Depth First Search (DFS)**.

2.2. Breadth First Search (BFS)

Algoritma Breadth First Search (BFS) adalah algoritma traversal graf yang mengeksplorasi simpul secara melebar atau *level-order*. Algoritma memulai pencarian pertama dengan mencari simpul awal diikuti dengan simpul-simpul yang berdekatan dengannya (tetangga). Setelah semua simpul yang bertetangga dengan simpul awal telah dikunjungi, algoritma akan mengunjungi simpul berikutnya yang belum dikunjungi namun bertetangga dengan simpul yang telah dikunjungi hingga semua simpul dalam graf telah dikunjungi. Urutan pengunjungan simpul dalam traversal graf secara BFS dapat digambarkan sebagai pohon penelusuran seperti berikut:



Gambar 2.2 Contoh hasil pohon penelusuran graf secara Breadth First Search

BFS bekerja dengan struktur data **Antrian (Queue)** yang beroperasi dengan prinsip **First-in-First-out (FIFO)**. Berikut adalah pseudocode lengkap dari algoritma BFS:

```

procedure BFS(input v:integer)
{ Traversal graf dengan algoritma pencarian BFS.

Masukan: v adalah simpul awal kunjungan
Keluaran: semua simpul yang dikunjungi dicetak ke layar
}
Deklarasi
w : integer
q : antrian;

procedure BuatAntrian(input/output q : antrian)
{ membuat antrian kosong, kepala(q) diisi 0 }

procedure MasukAntrian(input/output q:antrian, input v:integer)
{ memasukkan v ke dalam antrian q pada posisi belakang }

procedure HapusAntrian(input/output q:antrian, output v:integer)
{ menghapus v dari kepala antrian q }

function AntrianKosong(input q:antrian) → boolean
{ true jika antrian q kosong, false jika sebaliknya }

Algoritma:
BuatAntrian(q)          { buat antrian kosong }

write(v)                { cetak simpul awal yang dikunjungi }
dikunjungi[v]←true      { simpul v telah dikunjungi, tandai dengan
                        true}
MasukAntrian(q,v)       { masukkan simpul awal kunjungan ke dalam
                        antrian)

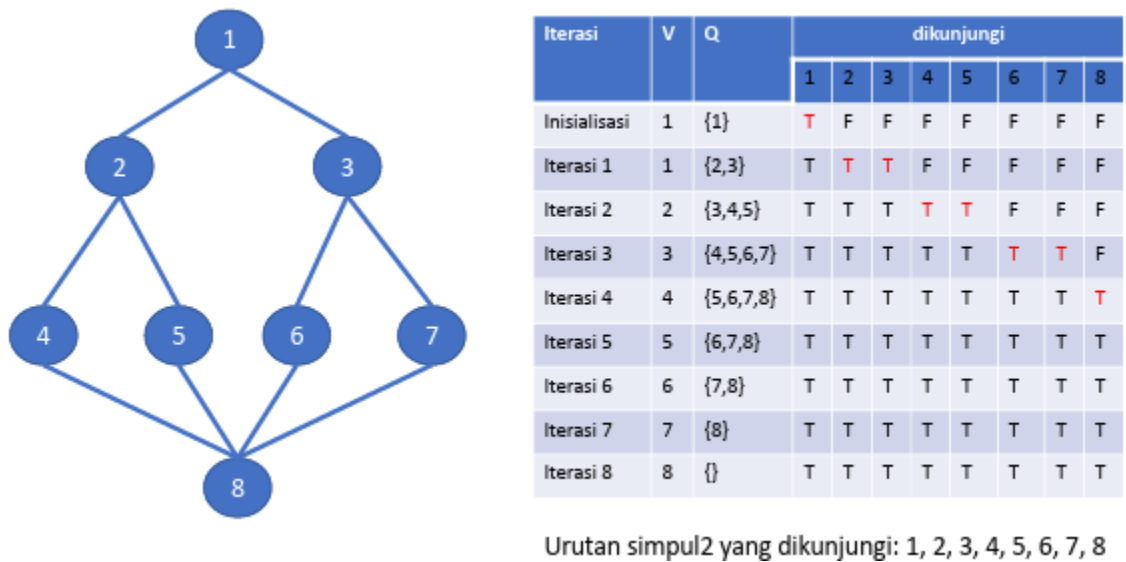
{ kunjungi semua simpul graf selama antrian belum kosong }
while not AntrianKosong(q) do
    HapusAntrian(q,v)    { simpul v telah dikunjungi, hapus dari
                        antrian }
    for tiap simpul w yang bertetangga dengan simpul v do
        if not dikunjungi[w] then
            write(w)      {cetak simpul yang dikunjungi}
            MasukAntrian(q,w)
            dikunjungi[w]←true
        endif
    endfor
endwhile
{ AntrianKosong(q) }

```

Gambar 2.3 Pseudocode Algoritma Breadth First Search

Dapat dilihat pada pseudocode di atas, BFS bergantung pada struktur data **Antrian/Queue (q: antrian)** sebagai pengatur lalu lintas penelusuran. Adapun terdapat array boolean **dikunjungi[v]** yang digunakan untuk menandai sebuah simpul yang sudah pernah diekspansi dengan **true** agar tidak dimasukkan ke dalam antrian lagi sehingga algoritma tidak berputar-putar tanpa henti. Algoritma dimulai dengan memasukkan simpul awal v ke dalam antrian dan ditandai dengan true. Selama antrian belum kosong (**while not AntrianKosong(q)**), algoritma akan mengeluarkan elemen paling depan (**HapusAntrian**). Algoritma kemudian mencari seluruh tetangga dari simpul yang baru dikeluarkan dan

jika tetangganya belum dikunjungi, tetangga tersebut akan diproses, ditandai, dan dilempar ke barisan paling belakang antrian (**MasukAntrian**). Adapun contoh ilustrasi dari pencarian solusi secara BFS yaitu sebagai berikut:



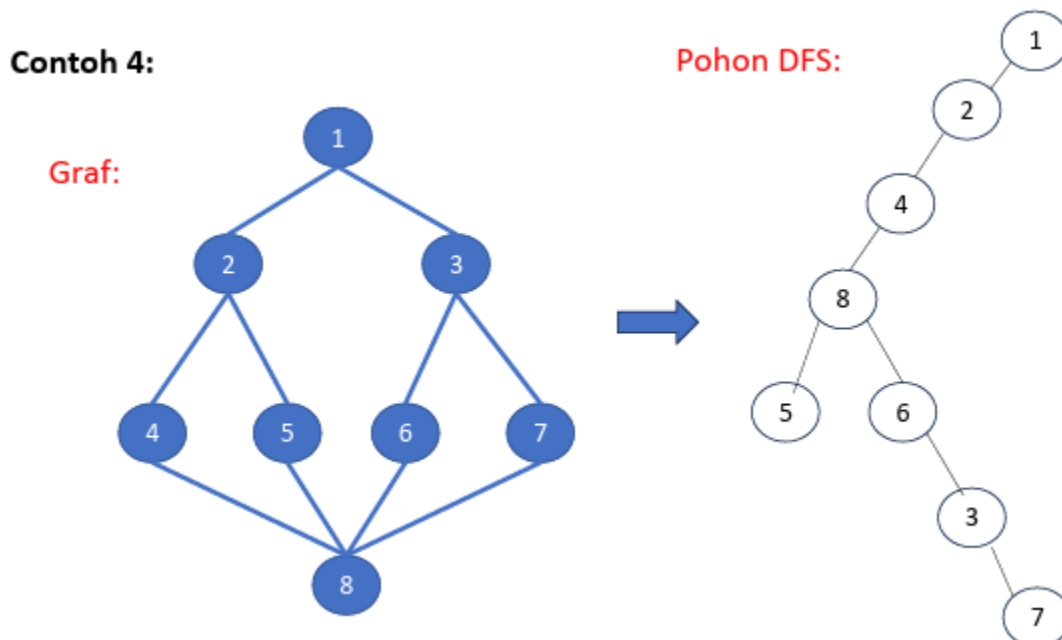
Gambar 2.4 Ilustrasi pencarian solusi secara Breadth First Search

Pada ilustrasi di atas, penelusuran dimulai dari simpul 1. Pada iterasi pertama, simpul 1 dikeluarkan dari antrian lalu tetangganya yaitu 2 dan 3 dimasukkan ke dalam antrian ($Q = \{2,3\}$). Pada iterasi 2, karena 2 masuk antrian lebih dulu daripada 3, maka 2 akan diproses duluan sehingga tetangganya yaitu 4 dan 5 dimasukkan ke belakang antrian ($Q = \{3,4,5\}$). Meskipun 4 dan 5 sudah ditemukan, algoritma akan memproses 3 terlebih dahulu dengan memasukkan 6 dan 7. Hal ini membuktikan bahwa BFS memproses satu level kedalaman terlebih dahulu sebelum memproses level berikutnya. Hasil urutan kunjungan simpul adalah 1, 2, 3, 4, 5, 6, 7, 8 yang terurut berdasarkan level kedalaman.

Algoritma BFS memiliki kompleksitas waktu sebesar $O(V + E)$ dimana V adalah jumlah simpul dan E adalah jumlah sisi. Sedangkan untuk kompleksitas ruangnya adalah $O(W)$ dengan W adalah lebar dari cabang pohon. Pada kasus terburuk, misalnya elemen HTML sangat datar dimana satu elemen memiliki ribuan anak (*child*), antrian (*queue*) akan menyimpan seluruh elemen di level tersebut secara bersamaan.

2.3. Depth First Search (DFS)

Algoritma Depth First Search (DFS) adalah algoritma traversal graf yang mengeksplorasi simpul secara mendalam atau *depth-order*. Tidak seperti algoritma BFS yang menelusuri semua simpul tetangga, DFS akan terus menelusuri hingga mencapai simpul yang tidak memiliki tetangga lagi. Pertama-tama, algoritma akan mengunjungi simpul awal kemudian mengunjungi simpul yang bertetangga dengannya. Proses ini akan terus diulangi hingga mencapai simpul yang sedemikian sehingga semua simpul yang bertetangga dengannya telah dikunjungi. Algoritma kemudian akan melakukan pencarian **runut-balik** (*backtracking*) ke simpul terakhir yang dikunjungi sebelumnya dan mempunyai simpul yang belum dikunjungi. Pencarian berakhir apabila tidak ada lagi simpul yang belum dikunjungi yang dapat dicapai dari simpul yang telah dikunjungi. Urutan pengunjungan simpul secara DFS dapat digambarkan menjadi pohon penelusuran seperti pada gambar berikut:



Gambar 2.5 Contoh hasil pohon penelusuran graf secara Depth First Search

DFS bekerja dengan struktur data **Tumpukan (Stack)** yang beroperasi dengan prinsip **Last-in-First-out (LIFO)**. Berikut adalah prosedur lengkap dari algoritma DFS:

```

procedure DFS(input v:integer)
  {Mengunjungi seluruh simpul graf dengan algoritma pencarian DFS}

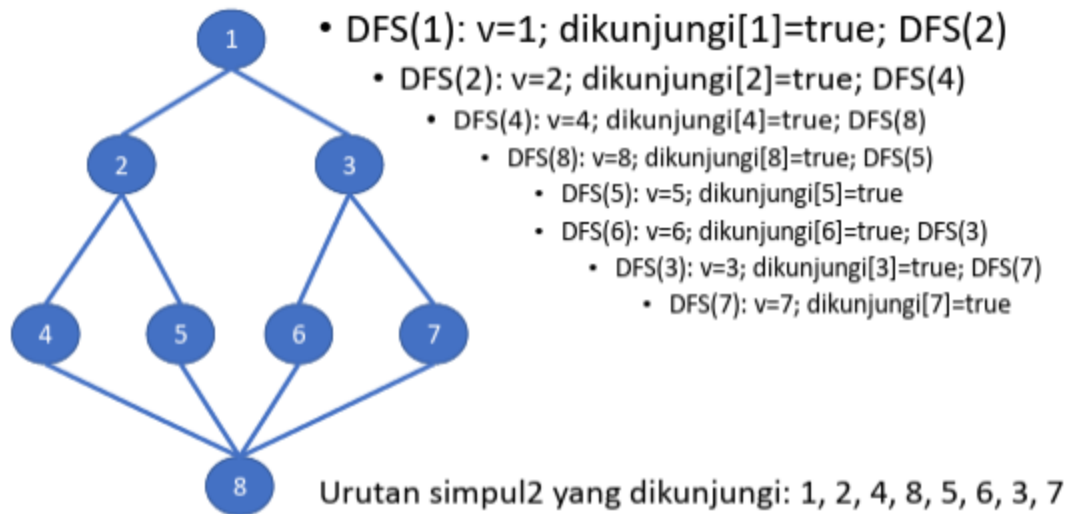
  Masukan: v adalah simpul awal kunjungan
  Keluaran: semua simpul yang dikunjungi ditulis ke layar
}
Deklarasi
  w : integer

Algoritma:
  write(v)
  dikunjungi[v]←true
  for w←1 to n do
    if A[v,w]=1 then (simpul v dan simpul w bertetangga )
      if not dikunjungi[w] then
        DFS(w)
      endif
    endif
  endfor

```

Gambar 2.6 Pseudocode Algoritma Depth First Search

Dapat dilihat pada pseudocode di atas bahwa algoritma DFS memiliki langkah yang jauh lebih ringkas karena memanfaatkan **Rekursi**. Berbeda dengan BFS, DFS tidak mendeklarasikan struktur data tambahan tetapi memanfaatkan **tumpukan (stack)** pemanggilan fungsi di memori untuk menyimpan jejak penelusuran. Bagian **if A[v,w]=1** menunjukkan bahwa algoritma mengecek keterhubungan antar simpul menggunakan **Adjacency matrix (matriks ketetanggaan)**. Jika bernilai 1 berarti ada jalan (sisi) dari simpul v ke simpul w. Saat algoritma menemukan satu tetangga w yang belum dikunjungi, perulangan akan dihentikan sementara dan algoritma akan memanggil **DFS(w)**. Algoritma kemudian melanjutkan sisa perulangannya setelah penelusuran telah sampai pada simpul yang tidak memiliki tetangga. Adapun ilustrasi dari pencarian solusi secara DFS yaitu sebagai berikut:



Gambar 2.7 Ilustrasi pencarian solusi secara DFS

Ilustrasi di atas menunjukkan bagaimana pemanggilan DFS secara rekursif pada graf yang sama dengan ilustrasi BFS sebelumnya. Penelusuran dimulai dari simpul 1 dan kemudian algoritma mengunjungi simpul 2. Setelah simpul 2, algoritma tidak mengunjungi simpul 3 seperti BFS melainkan turun ke tetangga pertama dari simpul 2 yaitu simpul 4 lalu turun ke tetangga selanjutnya yaitu simpul 8 hingga simpul 5. Pada saat sudah mencapai simpul 5, karena simpul 3 dan 8 sudah ditandai true, maka fungsi **DFS(5)** selesai dan algoritma akan *backtracking* ke fungsi pemanggilnya yaitu **DFS(8)**. Setelah mundur ke simpul 8, algoritma akan mengecek tetangga lainnya yang belum dikunjungi yaitu simpul 6. Dari simpul 6, selanjutnya akan dikunjungi simpul 3 lalu 7. Hasil urutan kunjungan simpul adalah 1, 2, 4, 8, 5, 6, 3, 7 yang terurut secara kedalaman cabang.

Untuk kompleksitas waktu dari DFS kurang lebih sama dengan BFS yaitu $O(V + E)$ karena setiap simpul dalam graf juga dikunjungi sebanyak satu kali. Sedangkan untuk kompleksitas ruangnya yaitu $O(H)$ dengan H merupakan kedalaman dari pohon dan algoritma ini cenderung lebih hemat memori dibandingkan BFS. Pada kasus terburuk, misalnya HTML bersarang dalam tanpa cabang, kompleksitas ruangnya bisa mencapai $O(N)$.

Bab 3

Aplikasi BFS dan DFS

3.1 Relevansi Algoritma Traversal pada Struktur DOM

Document Object Model (DOM) secara alami membentuk struktur pohon yang hierarkis, di mana setiap elemen HTML adalah sebuah simpul (node) dan hubungan antar elemen dinyatakan sebagai tepi (edge) antara parent dan child. Karena DOM adalah pohon, algoritma traversal graf seperti BFS dan DFS menjadi pilihan yang sangat tepat untuk digunakan dalam proses pencarian elemen.

Dalam konteks CSS selector, pencarian elemen tidak cukup hanya dengan menyimpan daftar elemen secara linear, melainkan harus mempertimbangkan posisi relatif sebuah elemen terhadap elemen lain di dalam pohon. Misalnya, selector seperti `div > p` mensyaratkan bahwa elemen `<p>` harus merupakan child langsung dari `<div>`, bukan sekadar keturunan jauh. Hal ini menjadikan traversal pohon bukan sekadar pilihan teknis, melainkan keharusan untuk dapat mengevaluasi hubungan struktural tersebut secara tepat.

Kedua algoritma, BFS maupun DFS, memiliki cara pandang yang berbeda dalam menjelajahi pohon. Perbedaan cara pandang inilah yang membuat keduanya menghasilkan perilaku pencarian yang berbeda pula, terutama ketika dikombinasikan dengan evaluasi CSS selector yang kompleks.

3.2 Strategi BFS untuk Pencarian Elemen

3.2.1. Prinsip Dasar

BFS menjelajahi pohon DOM secara berlapis, dimulai dari elemen root, kemudian mengunjungi seluruh elemen pada kedalaman pertama sebelum melanjutkan ke kedalaman berikutnya. Untuk menjaga urutan kunjungan tersebut, BFS menggunakan struktur data antrian (queue) dengan prinsip First In First Out (FIFO).

3.2.2. Alur Kerja BFS Sekuensial

Implementasi BFS pada aplikasi ini dimulai dengan memasukkan ID root ke dalam antrian. Pada setiap iterasi, satu node diambil dari depan antrian, kemudian dievaluasi apakah node tersebut cocok dengan query pencarian. Jika cocok, penelusuran langsung dihentikan dan hasil dikembalikan. Jika tidak cocok, seluruh child dari node tersebut dimasukkan ke belakang antrian untuk diproses pada iterasi berikutnya.

```
BFS(tree, query):  
    queue ← [root]  
    while queue tidak kosong:  
        node ← queue.dequeue()  
        if matches(node, query):  
            return node  
        for each child in node.children:  
            queue.enqueue(child)  
    return null
```

Dari pseudocode singkat diatas, bisa dilihat kalau sifat BFS yang menjelajahi level per level membuatnya cenderung menemukan elemen yang berada lebih dekat ke root lebih cepat dibandingkan DFS, terutama ketika elemen target berada di kedalaman yang dangkal.

3.2.3. Alur Kerja BFS Konkuren

Pada BFS konkuren, seluruh node dalam satu level diproses secara paralel menggunakan beberapa thread. Setiap thread mendapatkan sebagian dari node pada level tersebut (chunk), kemudian masing-masing thread secara mandiri mengevaluasi apakah node dalam bagiannya cocok dengan query.

Hasil dari setiap thread dikumpulkan kembali dan diurutkan berdasarkan posisi aslinya dalam level, sehingga urutan traversal tetap konsisten dengan BFS sekuensial. Jika setelah penggabungan hasil ditemukan node yang cocok, penelusuran dihentikan dan tidak melanjutkan ke level berikutnya.

Jumlah thread yang digunakan disesuaikan secara dinamis berdasarkan banyaknya node pada level tersebut dan jumlah thread yang tersedia pada sistem, sehingga tidak terjadi pemborosan sumber daya ketika jumlah node sedikit.

3.2.4 Karakteristik BFS

BFS memiliki beberapa karakteristik utama yang membedakannya dari algoritma traversal lain:

1. **Menggunakan antrian (queue) sebagai struktur data utama.** Setiap node yang akan dikunjungi dimasukkan ke belakang antrian, dan node yang diproses selalu diambil dari depan. Prinsip FIFO ini yang menjamin BFS berjalan level per level.
2. **Menjelajahi pohon secara melebar sebelum mendalami.** BFS tidak akan berpindah ke level berikutnya sebelum seluruh node pada level saat ini selesai dikunjungi. Akibatnya, urutan kunjungan selalu dari atas ke bawah dan dari kiri ke kanan dalam setiap level.
3. **Menjamin elemen yang ditemukan pertama adalah yang paling dangkal.** Karena BFS mengunjungi semua node di level k sebelum level $k+1$, elemen target yang pertama ditemukan

pasti memiliki kedalaman paling kecil di antara semua elemen yang cocok. Ini adalah sifat optimal BFS yang tidak dimiliki DFS.

4. **Konsumsi memori relatif lebih besar. Pada setiap iterasi**, BFS menyimpan seluruh node dari satu level sekaligus di dalam antrian. Semakin lebar pohon DOM (semakin banyak elemen pada satu level), semakin besar pula memori yang dibutuhkan.
5. **Lebih efisien ketika elemen target berada dekat dengan root**. Jika elemen yang dicari ada di level atas pohon, BFS akan menemukannya dengan sangat cepat tanpa perlu menyelami cabang-cabang yang dalam.

3.3 Strategi DFS untuk Pencarian Elemen

3.3.1 Prinsip Dasar

DFS menjelajahi pohon DOM dengan cara menyelami satu cabang sedalam-dalamnya terlebih dahulu sebelum berpindah ke cabang berikutnya. Implementasi pada aplikasi ini menggunakan pendekatan iteratif dengan struktur data tumpukan (stack) berprinsip Last In First Out (LIFO), bukan rekursif, untuk menghindari kemungkinan stack overflow pada pohon DOM yang sangat dalam.

3.3.2 Alur Kerja DFS Sekuensial

```
DFS(tree, query):  
    stack ← [root]  
    while stack tidak kosong:  
        node ← stack.pop()  
        if matches(node, query):  
            return node  
        for each child in reverse(node.children):  
            stack.push(child)  
    return null
```

DFS dimulai dengan memasukkan ID root ke dalam stack. Pada setiap iterasi, node paling atas diambil dari stack dan dievaluasi. Jika cocok, penelusuran dihentikan. Jika tidak, seluruh child dari node tersebut dimasukkan ke dalam stack dalam urutan terbalik, sehingga child pertama (paling kiri) diproses lebih dahulu sesuai urutan DFS pre-order dari kiri ke kanan.

3.3.3 Alur Kerja DFS Konkuren

Pada DFS konkuren, node root dievaluasi terlebih dahulu secara tunggal. Apabila root tidak cocok, setiap child langsung dari root menjadi titik awal subtree yang independen. Masing-masing subtree

tersebut kemudian ditelusuri secara paralel oleh thread yang berbeda, di mana setiap thread menjalankan DFS sekuensial penuh pada subtree yang menjadi tanggung jawabnya.

Setelah seluruh thread selesai, hasilnya digabungkan kembali dengan urutan yang mengikuti posisi child pada root, sehingga konsistensi urutan DFS dari kiri ke kanan tetap terjaga. Jika sebuah thread menemukan hasil, thread-thread berikutnya (yang menangani subtree di sebelah kanan) tidak perlu lagi diperiksa hasilnya.

Berbeda dengan BFS konkuren yang membagi pekerjaan per level, DFS konkuren membagi pekerjaan per subtree. Hal ini membuat DFS konkuren lebih efisien ketika pohon memiliki banyak cabang besar yang dapat diproses secara benar-benar independen.

3.3.4 Karakteristik DFS

DFS juga memiliki sejumlah karakteristik yang membentuk perilakunya dalam penelusuran pohon DOM:

1. **Menggunakan tumpukan (stack) sebagai struktur data utama.** Implementasi pada aplikasi ini menggunakan stack secara iteratif, bukan rekursif, untuk menghindari risiko stack overflow pada pohon DOM yang memiliki kedalaman sangat besar.
2. **Menjelajahi pohon secara mendalam sebelum melebar.** DFS akan terus menyelami satu cabang hingga ujung (leaf node) sebelum mundur dan berpindah ke cabang berikutnya. Urutan kunjungan mengikuti pola pre-order dari kiri ke kanan.
3. **Tidak menjamin elemen yang ditemukan pertama adalah yang paling dangkal.** DFS bisa saja melewati node yang cocok di cabang kanan karena sedang menyelami cabang kiri lebih dulu. Oleh karena itu, elemen yang ditemukan pertama oleh DFS belum tentu merupakan elemen dengan kedalaman terkecil.
4. **Konsumsi memori relatif lebih kecil.** Stack pada DFS hanya menyimpan node-node yang berada pada jalur aktif dari root ke node yang sedang dikunjungi. Jumlah node dalam stack maksimal sebanding dengan kedalaman pohon, bukan lebarnya.
5. **Lebih efisien ketika elemen target berada jauh di dalam satu cabang tertentu.** Jika elemen yang dicari terletak di ujung salah satu cabang, DFS dapat menemukannya dengan cepat jika cabang tersebut kebetulan ditelusuri lebih dulu, tanpa perlu memproses seluruh level di atasnya seperti yang dilakukan BFS.

3.4 Pendekatan Pencarian CSS Selector

3.4.1 Struktur SearchQuery

Untuk mendukung berbagai jenis CSS selector, implementasi ini mendefinisikan sebuah enum SearchQuery yang merangkum empat jenis query pencarian:

1. **Tag**: Mencari elemen berdasarkan nama tag HTML, misalnya div atau p.
2. **Attr**: Mencari elemen berdasarkan pasangan atribut dan nilai, misalnya class="container".
3. **TagAndAttr**: Kombinasi antara tag dan atribut secara bersamaan.
4. **Selector**: Pencarian menggunakan CSS selector penuh yang dapat mengandung combinator, seperti div > p.active atau ul ~ ol.

3.4.2 Evaluasi Query pada Setiap Node

Setiap node yang dikunjungi selama traversal dievaluasi menggunakan metode `matches_at`, yang menerima referensi ke pohon, indeks LCA, ID node, dan data node. Untuk query bertipe Tag dan *Attr*, evaluasi cukup membandingkan tag atau daftar atribut node secara langsung. Untuk query bertipe Selector, evaluasi diteruskan ke fungsi `query_matches_node` yang menangani parsing dan evaluasi combinator secara rekursif. Fungsi ini memanfaatkan indeks LCA untuk mengevaluasi hubungan antar-node, misalnya untuk memverifikasi apakah sebuah node merupakan ancestor dari node lain (digunakan dalam Descendant Combinator) atau apakah dua node berbagi parent yang sama (digunakan dalam Sibling Combinator).

3.4.3 Combinator yang Didukung

Combinator adalah mekanisme dalam CSS selector untuk menyatakan hubungan struktural antara dua elemen dalam pohon DOM. Aplikasi ini mendukung empat jenis combinator berikut:

1. **Child Combinator (>)**. Combinator ini mensyaratkan bahwa elemen target harus merupakan child langsung dari elemen sebelumnya dalam selector. Evaluasi dilakukan dengan memeriksa apakah parent langsung dari node yang sedang dikunjungi cocok dengan bagian selector di sebelah kiri tanda >. Jika parent langsung tidak cocok, node tersebut langsung gugur meskipun ancestor yang lebih jauh cocok.
2. **Descendant Combinator (*spasi*)**. Combinator ini lebih longgar dibandingkan Child Combinator, karena elemen target hanya perlu memiliki salah satu ancestor yang cocok dengan bagian selector di sebelah kiri, tidak harus parent langsung. Evaluasi memanfaatkan indeks LCA untuk memeriksa apakah terdapat ancestor yang memenuhi syarat di sepanjang jalur dari node tersebut ke root.
3. **Adjacent Sibling Combinator (+)**. Combinator ini mensyaratkan bahwa elemen target harus memiliki sibling yang tepat berada di sebelah kirinya (sibling yang langsung mendahuluinya) dan sibling tersebut cocok dengan bagian selector di sebelah kiri tanda +. Evaluasi dilakukan dengan mengambil daftar children dari parent node yang bersangkutan, lalu memeriksa node yang posisinya tepat satu sebelum node target.

4. **General Sibling Combinator (~).** Combinator ini lebih longgar dibandingkan Adjacent Sibling, karena elemen target hanya perlu memiliki setidaknya satu sibling di sebelah kirinya yang cocok, tidak harus yang tepat bersebelahan. Evaluasi dilakukan dengan memeriksa semua sibling yang berada di sebelah kiri node target dalam daftar children parent yang sama.

3.5 Pemanfaatan Multithreading

3.5.1 Motivasi

Pohon DOM dari halaman web dunia nyata dapat memiliki ribuan node. Penelusuran secara sekuensial pada pohon sebesar itu dapat memakan waktu yang terasa signifikan bagi pengguna. Dengan memanfaatkan beberapa core prosesor melalui multithreading, proses traversal dapat dilakukan secara paralel sehingga waktu pencarian berkurang.

3.5.2 Strategi Pembagian Kerja

Implementasi menggunakan `thread::scope` dari standar library Rust, yang menjamin bahwa semua thread selesai sebelum fungsi mengembalikan hasil. Jumlah thread yang dibuat tidak dipakukan pada suatu angka tetap, melainkan disesuaikan secara dinamis menggunakan `thread::available_parallelism()`, yaitu mengambil jumlah core logis yang tersedia di mesin yang menjalankan program.

Pada BFS konkuren, pembagian kerja dilakukan per level: node-node dalam satu level dibagi rata ke sejumlah thread. Pada DFS konkuren, pembagian kerja dilakukan per subtree: setiap child root ditangani oleh satu thread independen.

3.5.3 Menjaga Konsistensi Urutan

Karena thread dapat selesai dalam urutan yang tidak dapat diprediksi, hasil dari setiap thread diberi tanda posisi (*field position*) sebelum diproses. Setelah semua thread selesai, hasil digabungkan dan diurutkan berdasarkan posisi tersebut sebelum diteruskan ke tahap pencocokan. Cara ini memastikan bahwa meskipun proses berjalan paralel, urutan traversal yang dilaporkan ke pengguna tetap identik dengan urutan traversal sekuensial.

3.6 Pemanfaatan LCA Binary Lifting

3.6.1 Motivasi

Beberapa CSS selector seperti Descendant Combinator (*spasi*) dan General Sibling Combinator (~) memerlukan pengetahuan tentang hubungan hirarkis antara dua node dalam pohon. Tanpa struktur

bantu, memeriksa apakah node A adalah ancestor dari node B mengharuskan penelusuran dari B ke atas hingga root, yang memiliki kompleksitas $O(d)$ dengan d adalah kedalaman node.

LCA (Lowest Common Ancestor) dengan teknik Binary Lifting memungkinkan pengecekan hubungan ancestor dilakukan dengan lebih efisien, yaitu setelah fase pra komputasi $O(n \log n)$, setiap query LCA dapat dijawab dalam $O(\log n)$.

3.6.2 Struktur Data

Binary Lifting menyimpan tabel dua dimensi $up[node][k]$, di mana $up[node][k]$ menyimpan indeks ancestor ke- 2^k dari node. Tabel ini dibangun sekali saat awal traversal menggunakan DFS/BFS dari root, kemudian digunakan berulang kali sepanjang proses evaluasi selector.

3.6.3 Proses Pembangunan Indeks LCA

Indeks LCA dibangun dengan cara berikut. Pertama, dilakukan DFS dari root untuk menetapkan $up[node][0]$ (parent langsung) dan kedalaman setiap node. Setelah itu, tabel diisi secara iteratif untuk setiap level k dari 1 hingga $\log n$:

$$up[node][k] = up[up[node][k-1]][k-1]$$

Artinya, ancestor ke- 2^k dari sebuah node adalah ancestor ke- 2^{k-1} dari ancestor ke- 2^{k-1} node tersebut.

3.6.4 Penggunaan dalam Evaluasi Selector

Fungsi `is_ancestor_id` memanfaatkan `lca_id` untuk memeriksa apakah suatu node adalah ancestor dari node lain dengan cara, jika LCA dari node A dan node B adalah A itu sendiri, maka A adalah ancestor dari B. Fungsi inilah yang digunakan saat mengevaluasi Descendant Combinator dalam CSS selector.

3.7 Perbandingan BFS dan DFS dalam Konteks DOM

3.7.1 Perbedaan Hasil Pencarian

Karena BFS menjelajahi pohon level per level, BFS selalu menemukan elemen yang pertama muncul secara "horizontal" dalam DOM, yaitu elemen dengan kedalaman paling kecil di antara semua elemen yang cocok. Sementara itu, DFS menemukan elemen yang pertama ditemukan secara "vertikal", yaitu elemen yang paling cepat dijangkau ketika menelusuri cabang pertama secara mendalam.

Perilaku ini membuat BFS dan DFS dapat menghasilkan elemen yang berbeda ketika terdapat lebih dari satu elemen yang cocok dengan selector yang diberikan.

3.7.2 Perbandingan Performa

Performa relatif antara BFS dan DFS sangat bergantung pada posisi elemen target dalam pohon:

1. Jika elemen target berada di dekat root atau di level atas, BFS cenderung lebih cepat karena langsung menemukan elemen pada level tersebut tanpa perlu menyelami cabang-cabang terlebih dahulu.
2. Jika elemen target berada jauh di dalam salah satu cabang, DFS cenderung lebih cepat karena langsung menyelami cabang tersebut, sementara BFS harus memproses semua node pada setiap level di atasnya terlebih dahulu.
3. Pada kasus terburuk (elemen tidak ditemukan atau berada di posisi paling terakhir), keduanya mengunjungi seluruh node pohon sehingga kompleksitasnya sama, yaitu $O(n)$.

3.7.3 Konsumsi Memori

BFS menyimpan seluruh node pada satu level dalam antrian secara bersamaan. Pada pohon yang lebar (banyak node per level), konsumsi memori BFS bisa jauh lebih besar dibandingkan DFS. DFS hanya menyimpan node-node yang ada pada jalur aktif dari root ke node yang sedang dikunjungi, sehingga konsumsi memori maksimalnya sebanding dengan kedalaman pohon.

Bab 4

Implementasi dan Pengujian

4.1 Implementasi

4.1.1. Breadth First Search

```
use std::collections::VecDeque;
use std::thread;
use std::time::Instant;
use crate::modules::domnode::DomNode;
use crate::modules::selector::{query_matches_node, SelectorQuery};
use crate::modules::tree::{LcaBinaryLifting, Tree};

#[derive(Debug, Clone)]
pub enum SearchQuery {
    Tag(String),
    Attr(String, String),
    TagAndAttr { tag: String, key: String, value: String },
    Selector(SelectorQuery),
}

impl SearchQuery {
    pub fn matches_at(
        &self,
        tree: &Tree,
        lca: &LcaBinaryLifting,
        node_id: usize,
        node: &DomNode,
    ) -> bool {
        match self {
            SearchQuery::Tag(tag) => node.tag() == tag,
            SearchQuery::Attr(key, value) => {
                node.attrs().iter().any(|(k, v)| k == key && v == value)
            },
            SearchQuery::TagAndAttr { tag, key, value } => {
                node.tag() == tag
                && node.attrs().iter().any(|(k, v)| k == key && v ==
value)
            },
            SearchQuery::Selector(selector_query) => {
                query_matches_node(tree, lca, node_id, selector_query)
            }
        }
    }
}
```

```

    }
}

#[derive(Debug, Clone)]
pub struct TraversalStep {
    pub step: usize,
    pub node_id: usize,
    pub tag: String,
    pub depth: usize,
    pub matched: bool,
}

#[derive(Debug, Clone)]
pub struct TraversalMetrics {
    pub visited_nodes: usize,
    pub elapsed_ms: u128,
    pub max_depth: usize,
}

#[derive(Debug, Clone)]
pub struct TraversalResult {
    pub found: Option<DomNode>,
    pub found_id: Option<usize>,
    pub traversal_order: Vec<usize>,
    pub path_to_found: Vec<usize>,
    pub log: Vec<TraversalStep>,
    pub metrics: TraversalMetrics,
}

pub fn path_to_root(tree: &Tree, target_id: usize) -> Vec<usize> {
    let mut path = Vec::new();
    let mut current = Some(target_id);

    while let Some(node_id) = current {
        path.push(node_id);
        let index = match tree.find_index_from_id(node_id) {
            Some(i) => i,
            None => break,
        };
        current = tree.nodes[index].parent();
    }

    path.reverse();
}

```

```

    path
}

pub fn bfs(tree: &Tree, query: &SearchQuery) -> TraversalResult {
    let started_at = Instant::now();
    let lca_index = tree.build_lca_index();
    let mut queue: VecDeque<usize> = VecDeque::new();
    let mut traversal_order = Vec::new();
    let mut log = Vec::new();
    let mut found_id = None;

    queue.push_back(tree.root);

    while let Some(current_node_id) = queue.pop_front() {
        let current_index = match
tree.find_index_from_id(current_node_id) {
            Some(index) => index,
            None => continue,
        };

        let current_node = &tree.nodes[current_index];
        let matched = query.matches_at(tree, &lca_index,
current_node_id, current_node);

        traversal_order.push(current_node_id);
        log.push(TraversalStep {
            step: traversal_order.len(),
            node_id: current_node_id,
            tag: current_node.tag().to_string(),
            depth: current_node.depth(),
            matched,
        });

        if matched {
            found_id = Some(current_node_id);
            break;
        }

        for child_id in current_node.children() {
            queue.push_back(*child_id);
        }
    }

    let found = found_id.and_then(|id| {

```

```

        tree.find_index_from_id(id)
            .and_then(|index| tree.nodes.get(index).cloned())
    });

    let path_to_found = found_id
        .map(|id| path_to_root(tree, id))
        .unwrap_or_default();

    TraversalResult {
        found,
        found_id,
        traversal_order: traversal_order.clone(),
        path_to_found,
        log,
        metrics: TraversalMetrics {
            visited_nodes: traversal_order.len(),
            elapsed_ms: started_at.elapsed().as_millis(),
            max_depth: tree.max_depth(),
        },
    }
}

#[derive(Debug)]
struct LevelVisit {
    position: usize,
    node_id: usize,
    tag: String,
    depth: usize,
    matched: bool,
    children: Vec<usize>,
}

pub fn bfs_concurrent(tree: &Tree, query: &SearchQuery) ->
    TraversalResult {
    let started_at = Instant::now();
    let lca_index = tree.build_lca_index();
    let mut current_level: Vec<usize> = vec![tree.root];
    let mut traversal_order = Vec::new();
    let mut log = Vec::new();
    let mut found_id = None;

    while !current_level.is_empty() {
        let available_workers = thread::available_parallelism()
            .map(|n| n.get())
    }

```

```

        .unwrap_or(1);
let workers = available_workers.min(current_level.len()).max(1);
let chunk_size = current_level.len().div_ceil(workers);
let lca_ref = &lca_index;
let query_ref = query;

let mut level_visits: Vec<LevelVisit> = Vec::new();

thread::scope(|scope| {
    let mut handles = Vec::new();

    for (chunk_idx, chunk) in
current_level.chunks(chunk_size).enumerate() {
        let base_position = chunk_idx * chunk_size;

        handles.push(scope.spawn(move || {
            let mut partial = Vec::new();

            for (offset, node_id) in
chunk.iter().copied().enumerate() {
                let position = base_position + offset;

                let index = match
tree.find_index_from_id(node_id) {
                    Some(i) => i,
                    None => continue,
                };

                let node = &tree.nodes[index];
                partial.push(LevelVisit {
                    position,
                    node_id,
                    tag: node.tag().to_string(),
                    depth: node.depth(),
                    matched: query_ref.matches_at(tree, lca_ref,
node_id, node),
                    children: node.children().clone(),
                });
            }

            partial
        }));
    }
})

```



```

        for handle in handles {
            let mut partial = handle.join().expect("bfs worker
thread panicked");
            level_visits.append(&mut partial);
        }
    });

    level_visits.sort_by_key(|visit| visit.position);

    let mut next_level = Vec::new();
    let mut stop = false;

    for visit in level_visits {
        traversal_order.push(visit.node_id);
        log.push(TraversalStep {
            step: traversal_order.len(),
            node_id: visit.node_id,
            tag: visit.tag,
            depth: visit.depth,
            matched: visit.matched,
        });

        if visit.matched {
            found_id = Some(visit.node_id);
            stop = true;
            break;
        }

        next_level.extend(visit.children);
    }

    if stop {
        break;
    }

    current_level = next_level;
}

let found = found_id.and_then(|id| {
    tree.find_index_from_id(id)
        .and_then(|index| tree.nodes.get(index).cloned())
});

let path_to_found = found_id

```

```

        .map(|id| path_to_root(tree, id))
        .unwrap_or_default();

TraversalResult {
    found,
    found_id,
    traversal_order: traversal_order.clone(),
    path_to_found,
    log,
    metrics: TraversalMetrics {
        visited_nodes: traversal_order.len(),
        elapsed_ms: started_at.elapsed().as_millis(),
        max_depth: tree.max_depth(),
    },
}
}

```

Dalam implementasi `bfs.rs`, kami memanfaatkan beberapa struktur dengan yang paling terpenting untuk menampilkan *output* dari hasil `bfs()` adalah dari struktur `TraversalResult`, yang akan menyimpan *node* yang ditemukan, *id* dari *node* tersebut, urutan semua *node* yang telah dikunjungi, jalur dari *root* ke *node* yang ditemukan, *log* dari langkah-langkah yang diambil, dan statistik evaluasi dalam bentuk *metrics*. Secara ringkas, implementasi BFS akan memiliki alur yang tidak jauh berbeda dengan apa yang telah dijelaskan sebelumnya di bagian 2.2, berupa:

1. Masukkan *root node* ke antrian `VecDeque`.
2. Selama antrian tidak kosong, *node* terdepan akan diambil, dicek kecocokannya, dan catat ke `traversal_order` dan `log`.
 - a. Jika cocok, atau sesuai dengan permintaan *user*, pencarian akan diberhentikan dan menyimpan hasilnya sebagai `found_id`.
 - b. Jika belum cocok, atau belum sesuai dengan permintaan *user*, seluruh *children node* dari *node* yang sedang dieksplorasi akan dimasukkan ke dalam *queue*.
3. Return `TraversalResult` yang berisikan urutan *traversal*, *node* yang ditemukan, jalur ke *node*, serta metrik evaluasinya.

Implementasi dari *approach* BFS tersebut umumnya dilakukan secara sekuensial atau satu per satu sebelum melanjutkan ke *node* selanjutnya. Meskipun efektif, namun *approach* ini tidak efisien dan memakan waktu yang relatif lama, terutama pada *case* yang melihat banyak sekali *node*. Sebagai pemanfaatan lebih efisien dari *resource* komputasi, kami mengimplementasi `bfs_concurrent`. Secara ringkas, ini adalah versi BFS yang memparalelkan pemrosesan *node* pada

sebuah *level*. Setiap *level* dari *tree* akan diproses secara bersamaan oleh beberapa *thread* yang berbeda. Setelah semua selesai, hasil yang didapat akan dikumpulkan dan diurutkan ulang berdasarkan posisi aslinya seakan pemrosesan tidak berbeda dengan BFS sekuensial. *Approach* ini menguntungkan dalam efisiensi program dengan memanfaatkan *resource* lebih baik.

4.1.2. Depth First Search

```
use std::thread;
use std::time::Instant;
use crate::modules::domnode::DomNode;
use crate::modules::selector::{query_matches_node, SelectorQuery};
use crate::modules::tree::{LcaBinaryLifting, Tree};

#[derive(Debug, Clone)]
pub enum SearchQuery {
    Tag(String),
    Attr(String, String),
    TagAndAttr { tag: String, key: String, value: String },
    Selector(SelectorQuery),
}

impl SearchQuery {
    pub fn matches_at(
        &self,
        tree: &Tree,
        lca: &LcaBinaryLifting,
        node_id: usize,
        node: &DomNode,
    ) -> bool {
        match self {
            SearchQuery::Tag(tag) => node.tag() == tag,
            SearchQuery::Attr(key, value) => {
                node.attrs().iter().any(|(k, v)| k == key && v == value)
            },
            SearchQuery::TagAndAttr { tag, key, value } => {
                node.tag() == tag
                && node.attrs().iter().any(|(k, v)| k == key && v ==
value)
            },
            SearchQuery::Selector(selector_query) => {
                query_matches_node(tree, lca, node_id, selector_query)
            }
        }
    }
}
```

```

    }
}

#[derive(Debug, Clone)]
pub struct TraversalStep {
    pub step: usize,
    pub node_id: usize,
    pub tag: String,
    pub depth: usize,
    pub matched: bool,
}

#[derive(Debug, Clone)]
pub struct TraversalMetrics {
    pub visited_nodes: usize,
    pub elapsed_ms: u128,
    pub max_depth: usize,
}

#[derive(Debug, Clone)]
pub struct TraversalResult {
    pub found: Option<DomNode>,
    pub found_id: Option<usize>,
    pub traversal_order: Vec<usize>,
    pub path_to_found: Vec<usize>,
    pub log: Vec<TraversalStep>,
    pub metrics: TraversalMetrics,
}

pub fn path_to_root(tree: &Tree, target_id: usize) -> Vec<usize> {
    let mut path = Vec::new();
    let mut current = Some(target_id);

    while let Some(node_id) = current {
        path.push(node_id);
        let index = match tree.find_index_from_id(node_id) {
            Some(i) => i,
            None => break,
        };
        current = tree.nodes[index].parent();
    }

    path.reverse();
    path
}

```

```

}

fn run_dfs_from_start(
    tree: &Tree,
    lca: &LcaBinaryLifting,
    query: &SearchQuery,
    start_node_id: usize,
) -> (Vec<usize>, Vec<TraversalStep>, Option<usize>) {
    let mut stack: Vec<usize> = vec![start_node_id];
    let mut traversal_order = Vec::new();
    let mut log = Vec::new();
    let mut found_id = None;

    while let Some(current_node_id) = stack.pop() {
        let current_index = match
tree.find_index_from_id(current_node_id) {
            Some(index) => index,
            None => continue,
        };

        let current_node = &tree.nodes[current_index];
        let matched = query.matches_at(tree, lca, current_node_id,
current_node);

        traversal_order.push(current_node_id);
        log.push(TraversalStep {
            step: traversal_order.len(),
            node_id: current_node_id,
            tag: current_node.tag().to_string(),
            depth: current_node.depth(),
            matched,
        });

        if matched {
            found_id = Some(current_node_id);
            break;
        }

        for child_id in current_node.children().iter().rev() {
            stack.push(*child_id);
        }
    }

    (traversal_order, log, found_id)
}

```

```

}

pub fn dfs(tree: &Tree, query: &SearchQuery) -> TraversalResult {
    let started_at = Instant::now();
    let lca_index = tree.build_lca_index();

    let (traversal_order, mut log, found_id) =
        run_dfs_from_start(tree, &lca_index, query, tree.root);

    for (idx, step) in log.iter_mut().enumerate() {
        step.step = idx + 1;
    }

    let found = found_id.and_then(|id| {
        tree.find_index_from_id(id)
            .and_then(|index| tree.nodes.get(index).cloned())
    });

    let path_to_found = found_id
        .map(|id| path_to_root(tree, id))
        .unwrap_or_default();

    TraversalResult {
        found,
        found_id,
        traversal_order: traversal_order.clone(),
        path_to_found,
        log,
        metrics: TraversalMetrics {
            visited_nodes: traversal_order.len(),
            elapsed_ms: started_at.elapsed().as_millis(),
            max_depth: tree.max_depth(),
        },
    }
}

#[derive(Debug)]
struct SubtreeVisit {
    position: usize,
    traversal_order: Vec<usize>,
    log: Vec<TraversalStep>,
    found_id: Option<usize>,
}

```

```

pub fn dfs_concurrent(tree: &Tree, query: &SearchQuery) ->
TraversalResult {
    let started_at = Instant::now();
    let lca_index = tree.build_lca_index();

    let root_index = match tree.find_index_from_id(tree.root) {
        Some(index) => index,
        None => {
            return TraversalResult {
                found: None,
                found_id: None,
                traversal_order: Vec::new(),
                path_to_found: Vec::new(),
                log: Vec::new(),
                metrics: TraversalMetrics {
                    visited_nodes: 0,
                    elapsed_ms: started_at.elapsed().as_millis(),
                    max_depth: 0,
                },
            };
        }
    };

    let root_node = &tree.nodes[root_index];
    let root_matched = query.matches_at(tree, &lca_index, tree.root,
root_node);

    let mut traversal_order = vec![tree.root];
    let mut log = vec![TraversalStep {
        step: 1,
        node_id: tree.root,
        tag: root_node.tag().to_string(),
        depth: root_node.depth(),
        matched: root_matched,
    }];

    let mut found_id = if root_matched { Some(tree.root) } else { None
};

    if found_id.is_none() {
        let root_children = root_node.children().clone();
        let available_workers = thread::available_parallelism()
            .map(|n| n.get())
            .unwrap_or(1);
    }

```

```

let workers = available_workers.min(root_children.len()).max(1);
let chunk_size = root_children.len().div_ceil(workers);

let mut subtree_visits: Vec<SubtreeVisit> = Vec::new();

thread::scope(|scope| {
    let mut handles = Vec::new();

    for (chunk_idx, chunk) in
root_children.chunks(chunk_size).enumerate() {
        let base_position = chunk_idx * chunk_size;
        let lca_ref = &lca_index;
        let query_ref = query;

        handles.push(scope.spawn(move || {
            let mut partial = Vec::new();

            for (offset, node_id) in
chunk.iter().copied().enumerate() {
                let position = base_position + offset;
                let (sub_order, sub_log, sub_found_id) =
                    run_dfs_from_start(tree, lca_ref, query_ref,
node_id);

                partial.push(SubtreeVisit {
                    position,
                    traversal_order: sub_order,
                    log: sub_log,
                    found_id: sub_found_id,
                });
            }

            partial
        }));

        for handle in handles {
            let mut partial = handle.join().expect("dfs worker
thread panicked");
            subtree_visits.append(&mut partial);
        }
    });

    subtree_visits.sort_by_key(|visit| visit.position);

```



```

    for visit in subtree_visits {
        let start_step = traversal_order.len();

        traversal_order.extend(visit.traversal_order);
        for (idx, mut step) in visit.log.into_iter().enumerate() {
            step.step = start_step + idx + 1;
            log.push(step);
        }

        if found_id.is_none() {
            found_id = visit.found_id;
            if found_id.is_some() {
                break;
            }
        }
    }
}

let found = found_id.and_then(|id| {
    tree.find_index_from_id(id)
        .and_then(|index| tree.nodes.get(index).cloned())
});

let path_to_found = found_id
    .map(|id| path_to_root(tree, id))
    .unwrap_or_default();

TraversalResult {
    found,
    found_id,
    traversal_order: traversal_order.clone(),
    path_to_found,
    log,
    metrics: TraversalMetrics {
        visited_nodes: traversal_order.len(),
        elapsed_ms: started_at.elapsed().as_millis(),
        max_depth: tree.max_depth(),
    },
}
}

```

Dalam implementasi `dfs.rs`, kami memanfaatkan struktur `TraversalResult` kembali seperti yang ada pada implementasi dari BFS sebelumnya. Perbedaan dari implementasi DFS tersendiri hanya terletak pada teoritisnya saja, yang dimana DFS akan menggunakan *stack* (LIFO). Dalam kata lain, eksplorasi DFS akan merujuk lebih dalam (*depth*) baru menyamping. Seperti penjelasan sebelumnya, implementasi DFS akan memiliki alur yang tidak jauh berbeda dengan apa yang telah dijelaskan sebelumnya di bagian 2.3, berupa:

1. Masukkan *root node* ke *stack* `VecDeque`.
2. Selama *stack* tidak kosong, *node* paling atas akan diambil, dicek kecocokannya, dan catat ke `traversal_order` dan `log`.
 - a. Jika cocok, atau sesuai dengan permintaan *user*, pencarian akan diberhentikan dan menyimpan hasilnya sebagai `found_id`.
 - b. Jika belum cocok, atau belum sesuai dengan permintaan *user*, seluruh *children node* dari *node* yang sedang dieksplorasi akan dimasukkan ke dalam *stack* dengan urutan terbalik (Kanan $\rightarrow$ Kiri). Hal tersebut untuk mempertahankan urutan kiri menjadi yang paling pertama dieksplorasi.
3. Return `TraversalResult` yang berisikan urutan *traversal*, *node* yang ditemukan, jalur ke *node*, serta metrik evaluasinya.

Tidak jauh berbeda dengan BFS, implementasi DFS juga terdapat dalam bentuk paralel yang memanfaatkan beberapa *threads* yang menjalankan proses perhitungan yang sama dalam waktu yang sama. Hal yang membedakan adalah paralelisasi akan dilakukan per-*subtree* dari anak *root*. Jika nanti salah satu *subtree* sudah menemukan *node* yang sesuai dengan permintaan *user*, *subtree* berikut-berikutnya tidak akan perlu diproses lagi.

4.1.3. DomNode

```
#[derive(Debug, Clone)]
pub struct DomNode {
    id: usize,
    tag: String,
    parent: Option<usize>,
    children: Vec<usize>,
    attrs: Vec<(String, String)>,
    depth: usize
}

impl DomNode {
    pub fn new(id: usize, tag: String, parent: Option<usize>, attrs:
```

```

Vec<(String, String)>, depth: usize) -> Self {
    DomNode {
        id,
        tag,
        parent,
        children: Vec::new(),
        attrs,
        depth
    }
}

pub fn id(&self) -> usize {
    self.id
}

pub fn tag(&self) -> &str {
    &self.tag
}

pub fn parent(&self) -> Option<usize> {
    self.parent
}

pub fn children(&self) -> &Vec<usize> {
    &self.children
}

pub fn attrs(&self) -> &Vec<(String, String)> {
    &self.attrs
}

pub fn depth(&self) -> usize {
    self.depth
}

pub fn add_child(&mut self, child_id: usize) {
    self.children.push(child_id);
}
}

```

Untuk terkait representasi elemen HTML, kami mengimplementasi **domnode.rs** untuk hal tersebut. Setiap elemen HTML nantinya akan direpresentasi dalam sebuah *node* yang

memiliki struktur atau konten berupa *id*, *tag*, *parent*, *children*, *attrs*, dan *depth*. Terkait struktur tersebut, dapat dijelaskan dimana setiap *node* akan menyimpan sebuah *id* yang unik, nama *tag* HTML, referensi ke *parent*-nya, daftar *id* dari setiap *children*-nya, daftar atribut dengan format *key-value*, serta kedalaman *node* tersebut dalam keseluruhan pohon. Dengan hal tersebut, dapat terlihat bahwa implementasi kami tidak menggunakan *pointer* sebagai penghubung antar-*node*, melainkan memanfaatkan *id*. Kami juga mengimplementasi semacam *getter* dan *setter* untuk beberapa komponen dari *node*-nya.

4.1.4. Tree

```
use crate::modules::domnode::DomNode;
use std::collections::HashMap;
#[derive(Debug, Clone)]

pub struct Tree{
    pub nodes: Vec<DomNode>,
    pub root: usize,
    pub id_to_index: HashMap<usize, usize>,
}

#[derive(Debug, Clone)]
pub struct LcaBinaryLifting {
    up: Vec<Vec<Option<usize>>>>,
    depth: Vec<usize>,
}

impl Tree{

    pub fn new() -> Self {
        Self {
            nodes: Vec::new(),
            root: 0,
            id_to_index: HashMap::new(),
        }
    }

    pub fn find_index_from_id(&self, id:usize) -> Option<usize> {
        return self.id_to_index.get(&id).copied();
    }

    pub fn insert_node(&mut self, node: DomNode){
```

```

        let id = node.id();
        if self.id_to_index.contains_key(&id){
            return;
        }
        let index = self.nodes.len();
        self.nodes.push(node);
        self.id_to_index.insert(id, index);
    }

    pub fn add_root(&mut self, tag: String, attrs: Vec<(String,
String)>){
        if self.id_to_index.contains_key(&0){
            return;
        }
        self.insert_node(DomNode::new(0,tag, None, attrs, 0));
        self.root = 0;
    }

    pub fn add_child(&mut self, id: usize, tag: String, parent_id:usize,
attrs: Vec<(String, String)>){
        if self.id_to_index.contains_key(&id){
            return;
        }

        let parent_idx = match self.find_index_from_id(parent_id){
            Some(idx) => idx,
            None => return,
        };

        let parent_depth = self.nodes[parent_idx].depth();

        self.insert_node(DomNode::new(id,tag,Some(parent_id),attrs,parent_depth+
1));

        self.nodes[parent_idx].add_child(id);

    }

    pub fn max_depth(&self) -> usize {
        self.nodes.iter().map(|node| node.depth()).max().unwrap_or(0)
    }

    pub fn build_lca_index(&self) -> LcaBinaryLifting {
        LcaBinaryLifting::build(self)
    }

```

```

    }
}

impl LcaBinaryLifting {
    pub fn build(tree: &Tree) -> Self {
        let n = tree.nodes.len();
        if n == 0 {
            return Self {
                up: Vec::new(),
                depth: Vec::new(),
            };
        }

        let mut max_log = 1usize;
        while (1usize << max_log) <= n {
            max_log += 1;
        }

        let mut up = vec![vec![None; max_log]; n];
        let mut depth = vec![0usize; n];
        let mut visited = vec![false; n];

        if let Some(root_idx) = tree.find_index_from_id(tree.root) {
            let mut stack = vec![root_idx];
            visited[root_idx] = true;

            while let Some(parent_idx) = stack.pop() {
                let parent_depth = depth[parent_idx];

                for child_id in tree.nodes[parent_idx].children() {
                    let child_idx = match
tree.find_index_from_id(*child_id) {
                        Some(index) => index,
                        None => continue,
                    };

                    if visited[child_idx] {
                        continue;
                    }

                    visited[child_idx] = true;
                    up[child_idx][0] = Some(parent_idx);
                    depth[child_idx] = parent_depth + 1;
                    stack.push(child_idx);
                }
            }
        }
    }
}

```

```

    }
}

for node_idx in 0..n {
    if visited[node_idx] {
        continue;
    }

    up[node_idx][0] = tree.nodes[node_idx]
        .parent()
        .and_then(|parent_id|
tree.find_index_from_id(parent_id));
    }

    for jump in 1..max_log {
        for node_idx in 0..n {
            up[node_idx][jump] = up[node_idx][jump - 1]
                .and_then(|mid_idx| up[mid_idx][jump - 1]);
        }
    }

    Self { up, depth }
}

pub fn lca_index(&self, a_idx: usize, b_idx: usize) -> Option<usize>
{
    if self.up.is_empty() {
        return None;
    }

    if a_idx >= self.up.len() || b_idx >= self.up.len() {
        return None;
    }

    let mut a = a_idx;
    let mut b = b_idx;

    if self.depth[a] < self.depth[b] {
        std::mem::swap(&mut a, &mut b);
    }

    let diff = self.depth[a] - self.depth[b];
    for jump in (0..self.up[0].len()).rev() {

```

```

        if ((diff >> jump) & 1) == 1 {
            a = self.up[a][jump]?;
        }
    }

    if a == b {
        return Some(a);
    }

    for jump in (0..self.up[0].len()).rev() {
        let next_a = self.up[a][jump];
        let next_b = self.up[b][jump];

        if next_a.is_some() && next_a != next_b {
            a = next_a?;
            b = next_b?;
        }
    }

    self.up[a][0]
}

pub fn lca_id(&self, tree: &Tree, a_id: usize, b_id: usize) ->
Option<usize> {
    let a_idx = tree.find_index_from_id(a_id)?;
    let b_idx = tree.find_index_from_id(b_id)?;
    let lca_idx = self.lca_index(a_idx, b_idx)?;
    Some(tree.nodes[lca_idx].id())
}

pub fn is_ancestor_id(&self, tree: &Tree, ancestor_id: usize,
node_id: usize) -> bool {
    matches!(self.lca_id(tree, ancestor_id, node_id), Some(id) if id
== ancestor_id)
}
}

```

Struktur *tree* yang kami pakai untuk BFS & DFS dalam representasi *node-node* yang ada pada HTML sebuah *domain* akan berbentuk seperti di tree.rs. Struktur *tree* akan menyimpan seluruh *node* yang ada ke dalam sebuah `Vec<DomNode>`, alias sebuah vektor berisikan *node*, dan menggunakan *hashmap* dalam `HashMap<usize, usize>` sebagai indeks pencarian ID ke posisi

array. Latar belakang dari desain kami adalah mempertahankan kompleksitas di $O(1)$ saat *lookup* berdasarkan *id*. Selain itu, untuk mendukung pencocokan CSS *selector* yang melibatkan relasi *descendant* (e.g. `div p`), kami mengimplementasi *LcaBinaryLifting*. Nantinya, tabel `up[v][j]` akan menyimpan *parent* ke- 2^j dari *node* *v*. Dengan adanya tabel ini, *lowest common ancestor* (LCA) antara dua *node* dapat dihitung dengan relatif cepat dan efisien, yaitu sekitar $O(\log(n))$.

4.1.5. Parser

```
use crate::modules::tree::Tree;

const VOID_TAGS: [&str; 14] = [
    "area", "base", "br", "col", "embed", "hr", "img", "input", "link",
    "meta", "param", "source",
    "track", "wbr",
];

// fungsi parsing HTML menjadi Tree
pub fn parse_html_to_tree(html: &str) -> Result<Tree, String> {
    // menggunakan root sintetis untuk memastikan bahwa semua node
    // memiliki parent,
    // sehingga traversal dan pencocokan selector dapat dilakukan dengan
    // lebih konsisten,
    // bahkan untuk fragmen HTML yang tidak lengkap atau tidak valid
    let mut tree = Tree::new();
    tree.add_root("document".to_string(), Vec::new());

    let mut stack: Vec<usize> = vec![tree.root];
    let mut next_id = 1usize;
    let mut cursor = 0usize;

    while let Some(start_rel) = html[cursor..].find('<') {
        let start = cursor + start_rel;

        if html[start..].starts_with("<!--") {
            // skip comment untuk toleransi markup yang tidak sempurna
            // memastikan bahwa komentar yang dimulai dengan <!--
            // diakhiri dengan -->,
            // jika tidak ditemukan --> maka akan menganggap sisa string
            // sebagai komentar dan berhenti parsing
            if let Some(end_rel) = html[start + 4..].find("-->") {
                cursor = start + 4 + end_rel + 3;
                continue;
            }
        }
    }
```

```

        break;
    }

    let end = match html[start..].find('>') {
        Some(rel) => start + rel,
        None => break,
    };

    let raw_token = html[start + 1..end].trim();
    if raw_token.is_empty() {
        cursor = end + 1;
        continue;
    }

    if raw_token.starts_with('!') || raw_token.starts_with('?') {
        cursor = end + 1;
        continue;
    }

    if let Some(rest) = raw_token.strip_prefix('/') {
        let closing_tag = rest
            .split_whitespace()
            .next()
            .unwrap_or("")
            .to_ascii_lowercase();

        if !closing_tag.is_empty() {
            // pop stack sampai menemukan tag pembuka yang sesuai
            // dengan tag penutup,
            // untuk toleransi markup yang tidak sempurna
            while stack.len() > 1 {
                let current_id =
*stack.last().unwrap_or(&tree.root);
                let current_index = match
tree.find_index_from_id(current_id) {
                    Some(index) => index,
                    None => {
                        stack.pop();
                        continue;
                    }
                };
            };

            let current_tag =
tree.nodes[current_index].tag().to_ascii_lowercase();

```

```

        stack.pop();

        if current_tag == closing_tag {
            break;
        }
    }

    cursor = end + 1;
    continue;
}

let self_closing = raw_token.ends_with('/');
let normalized = if self_closing {
    raw_token[..raw_token.len() - 1].trim()
} else {
    raw_token
};

let (tag, attrs) = parse_opening_tag(normalized);
if tag.is_empty() {
    cursor = end + 1;
    continue;
}

let parent_id = *stack.last().unwrap_or(&tree.root);
tree.add_child(next_id, tag.clone(), parent_id, attrs);

// void/self-closing tags tidak dimasukkan ke stack karena tidak
memiliki children
if !self_closing && !is_void_tag(&tag) {
    stack.push(next_id);
}

next_id += 1;
cursor = end + 1;
}

Ok(tree)
}

// fungsi parsing opening tag untuk mengekstrak nama tag dan atributnya
// fungsi ini menangani berbagai format atribut, termasuk yang
menggunakan tanda kutip ganda, tunggal, atau tanpa tanda kutip

```

```

fn parse_opening_tag(input: &str) -> (String, Vec<(String, String)>) {
    let mut chars = input.chars().peekable();

    while matches!(chars.peek(), Some(c) if c.is_whitespace()) {
        chars.next();
    }

    let mut tag = String::new();
    while let Some(ch) = chars.peek().copied() {
        if ch.is_whitespace() {
            break;
        }
        tag.push(ch);
        chars.next();
    }

    let tag = tag.to_ascii_lowercase();
    let mut attrs = Vec::new();

    loop {
        while matches!(chars.peek(), Some(c) if c.is_whitespace()) {
            chars.next();
        }

        if chars.peek().is_none() {
            break;
        }

        let mut key = String::new();
        while let Some(ch) = chars.peek().copied() {
            if ch.is_whitespace() || ch == '=' {
                break;
            }
            key.push(ch);
            chars.next();
        }

        if key.is_empty() {
            chars.next();
            continue;
        }

        while matches!(chars.peek(), Some(c) if c.is_whitespace()) {
            chars.next();
        }
    }
}

```

```

    }

    let mut value = String::new();

    if matches!(chars.peek(), Some('=')) {
        chars.next();

        while matches!(chars.peek(), Some(c) if c.is_whitespace()) {
            chars.next();
        }

        if let Some(quote @ ('"' | '\'')) = chars.peek().copied() {
            // handle tanda kutip ganda atau tunggal untuk nilai
            // memastikan bahwa nilai yang diambil adalah apa yang
            // ada di dalam tanda kutip
            chars.next();

            while let Some(ch) = chars.peek().copied() {
                chars.next();
                if ch == quote {
                    break;
                }
                value.push(ch);
            }
        } else {
            // handle nilai atribut tanpa tanda kutip,
            // mengambil karakter sampai menemukan whitespace atau
            // akhir string
            while let Some(ch) = chars.peek().copied() {
                if ch.is_whitespace() {
                    break;
                }
                value.push(ch);
                chars.next();
            }
        }

        attrs.push((key.to_ascii_lowercase(), value));
    }

    (tag, attrs)
}

```

```
fn is_void_tag(tag: &str) -> bool {
    VOID_TAGS.iter().any(|t| t.eq_ignore_ascii_case(tag))
}
```

Untuk dapat menganalisis dan memproses data dari sebuah domain, kami mengimplementasikan sebuah *parser* yang secara ringkas akan melakukan *parse* dari HTML menjadi bentuk *tree*. *Parser* akan membaca *string* HTML karakter per karakter mencari tanda “<”. Setiap *token* antar <...> akan nantinya diidentifikasi sebagai:

- Komentar <!-- -->: Ini akan dilewati saja karena kurang penting.
- Tag <!DOCTYPE> atau <?...>: Ini akan dilewati saja karena kurang penting.
- Tag penutup </tag> yang akan melakukan *pop stack*.
- Tag pembuka <tag ...> yang akan membuat *node baru* dan *push* ke *stack*.

Terdapat beberapa fungsi pendukung seperti `parse_opening_tag()` untuk memecahkan *string* mentah menjadi nama *tag* dan daftar atribut, serta `is_void_tag()` untuk menangani *quotation*.

4.1.6. Selector

```
use crate::modules::domnode::DomNode;
use crate::modules::tree::{LcaBinaryLifting, Tree};

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Combinator {
    Child,
    Descendant,
    AdjacentSibling,
    GeneralSibling,
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum SimpleSelector {
    Universal,
    Tag(String),
    Class(String),
    Id(String),
    AttrEq { key: String, value: String },
}

#[derive(Debug, Clone)]
pub struct SelectorPart {
```

```

        pub selector: SimpleSelector,
        pub relation_to_left: Option<Combinator>,
    }

#[derive(Debug, Clone)]
pub struct SelectorQuery {
    pub parts_right_to_left: Vec<SelectorPart>,
}

impl SelectorQuery {
    pub fn parse(input: &str) -> Result<Self, String> {
        parse_selector(input)
    }
}

pub fn parse_selector(input: &str) -> Result<SelectorQuery, String> {
    let mut selectors_left_to_right: Vec<String> = Vec::new();
    let mut combinators_left_to_right: Vec<Combinator> = Vec::new();
    let mut buffer = String::new();
    let mut bracket_depth = 0usize;
    let mut pending_descendant = false;
    let mut last_was_selector = false;

    let flush_selector = |buffer: &mut String,
                        pending_descendant: &mut bool,
                        last_was_selector: &mut bool,
                        selectors: &mut Vec<String>,
                        combinators: &mut Vec<Combinator>| {
        let token = buffer.trim();
        if token.is_empty() {
            buffer.clear();
            return;
        }

        if *pending_descendant && *last_was_selector {
            combinators.push(Combinator::Descendant);
        }

        selectors.push(token.to_string());
        *last_was_selector = true;
        *pending_descendant = false;
        buffer.clear();
    };

```

```

for ch in input.chars() {
    match ch {
        '[' => {
            bracket_depth += 1;
            buffer.push(ch);
        }
        ']' => {
            if bracket_depth == 0 {
                return Err("Selector bracket mismatch".to_string());
            }
            bracket_depth -= 1;
            buffer.push(ch);
        }
        '>' | '+' | '~' if bracket_depth == 0 => {
            flush_selector(
                &mut buffer,
                &mut pending_descendant,
                &mut last_was_selector,
                &mut selectors_left_to_right,
                &mut combinators_left_to_right,
            );

            if !last_was_selector {
                return Err("Invalid combinator
placement".to_string());
            }

            let combinator = match ch {
                '>' => Combinator::Child,
                '+' => Combinator::AdjacentSibling,
                '~' => Combinator::GeneralSibling,
                _ => unreachable!(),
            };
            combinators_left_to_right.push(combinator);
            last_was_selector = false;
            pending_descendant = false;
        }
        c if c.is_whitespace() && bracket_depth == 0 => {
            flush_selector(
                &mut buffer,
                &mut pending_descendant,
                &mut last_was_selector,
                &mut selectors_left_to_right,
                &mut combinators_left_to_right,
            );
        }
    }
}

```



```

        );

        if last_was_selector {
            pending_descendant = true;
        }
    }
    _ => buffer.push(ch),
}

}

flush_selector(
    &mut buffer,
    &mut pending_descendant,
    &mut last_was_selector,
    &mut selectors_left_to_right,
    &mut combinators_left_to_right,
);

if bracket_depth != 0 {
    return Err("Selector bracket mismatch".to_string());
}

if selectors_left_to_right.is_empty() {
    return Err("Empty selector".to_string());
}

if combinators_left_to_right.len() + 1 !=
selectors_left_to_right.len() {
    return Err("Invalid selector chain".to_string());
}

let mut parts_right_to_left = Vec::new();
for selector_idx in (0..selectors_left_to_right.len()).rev() {
    let selector =
parse_simple_selector(&selectors_left_to_right[selector_idx])?;
    let relation_to_left = if selector_idx > 0 {
        Some(combinators_left_to_right[selector_idx - 1])
    } else {
        None
    };
};

parts_right_to_left.push(SelectorPart {
    selector,
    relation_to_left,

```

```

    });
}

Ok(SelectorQuery {
    parts_right_to_left,
})
}

pub fn query_matches_node(
    tree: &Tree,
    lca: &LcaBinaryLifting,
    node_id: usize,
    query: &SelectorQuery,
) -> bool {
    if query.parts_right_to_left.is_empty() {
        return false;
    }

    let mut current_id = node_id;

    for part_idx in 0..query.parts_right_to_left.len() {
        let part = &query.parts_right_to_left[part_idx];
        let current_node = match get_node(tree, current_id) {
            Some(node) => node,
            None => return false,
        };

        if !matches_simple_selector(current_node, &part.selector) {
            return false;
        }

        let relation = match part.relation_to_left {
            Some(rel) => rel,
            None => continue,
        };

        let left_part = match query.parts_right_to_left.get(part_idx +
1) {
            Some(p) => p,
            None => return false,
        };

        let next_id = match match_left_node(tree, lca, current_id,
relation, &left_part.selector) {

```

```

        Some(id) => id,
        None => return false,
    };

    current_id = next_id;
}

true
}

pub fn find_matching_nodes(tree: &Tree, query: &SelectorQuery) ->
Vec<usize> {
    let lca = tree.build_lca_index();
    tree.nodes
        .iter()
        .map(|node| node.id())
        .filter(|node_id| query_matches_node(tree, &lca, *node_id,
query))
        .collect()
}

fn parse_simple_selector(token: &str) -> Result<SimpleSelector, String>
{
    if token == "*" {
        return Ok(SimpleSelector::Universal);
    }

    if let Some(rest) = token.strip_prefix('.') {
        if rest.is_empty() {
            return Err("Class selector cannot be empty".to_string());
        }
        return Ok(SimpleSelector::Class(rest.to_string()));
    }

    if let Some(rest) = token.strip_prefix('#') {
        if rest.is_empty() {
            return Err("ID selector cannot be empty".to_string());
        }
        return Ok(SimpleSelector::Id(rest.to_string()));
    }

    if token.starts_with('[') && token.ends_with(']') {
        let content = &token[1..token.len() - 1];
        let mut parts = content.splitn(2, '=');

```

```

        let key = parts.next().unwrap_or("").trim();
        let value = parts.next().unwrap_or("").trim();

        if key.is_empty() || value.is_empty() {
            return Err("Attribute selector must be in [key=value]
format".to_string());
        }

        let normalized_value =
value.trim_matches('"').trim_matches('\''');
        return Ok(SimpleSelector::AttrEq {
            key: key.to_string(),
            value: normalized_value.to_string(),
        });
    }

    Ok(SimpleSelector::Tag(token.to_string()))
}

fn get_node(tree: &Tree, node_id: usize) -> Option<&DomNode> {
    tree.find_index_from_id(node_id)
        .and_then(|index| tree.nodes.get(index))
}

fn matches_simple_selector(node: &DomNode, selector: &SimpleSelector) ->
bool {
    match selector {
        SimpleSelector::Universal => true,
        SimpleSelector::Tag(tag) =>
node.tag().eq_ignore_ascii_case(tag),
        SimpleSelector::Class(class_name) => node
            .attrs()
            .iter()
            .find(|(key, _)| key.eq_ignore_ascii_case("class"))
            .map(|(_, value)| {
                value
                    .split_whitespace()
                    .any(|part| part.eq_ignore_ascii_case(class_name))
            })
            .unwrap_or(false),
        SimpleSelector::Id(id_value) => node
            .attrs()
            .iter()
            .find(|(key, _)| key.eq_ignore_ascii_case("id"))
    }
}

```

```

        .map(|(_, value)| value == id_value)
        .unwrap_or(false),
        SimpleSelector::AttrEq { key, value } =>
node.attrs().iter().any(|(k, v)| {
    k.eq_ignore_ascii_case(key) && v == value
})),
    }
}

fn match_left_node(
    tree: &Tree,
    lca: &LcaBinaryLifting,
    right_id: usize,
    relation: Combinator,
    left_selector: &SimpleSelector,
) -> Option<usize> {
    match relation {
        Combinator::Child => {
            let right_node = get_node(tree, right_id)?;
            let parent_id = right_node.parent()?;
            let parent_node = get_node(tree, parent_id)?;
            if matches_simple_selector(parent_node, left_selector) {
                Some(parent_id)
            } else {
                None
            }
        }
        Combinator::Descendant => {
            let mut current = get_node(tree, right_id)?.parent();
            while let Some(ancestor_id) = current {
                if !lca.is_ancestor_id(tree, ancestor_id, right_id) {
                    current = get_node(tree, ancestor_id).and_then(|n|
n.parent());

                    continue;
                }

                let ancestor_node = get_node(tree, ancestor_id)?;
                if matches_simple_selector(ancestor_node, left_selector)
{
                    return Some(ancestor_id);
                }

                current = ancestor_node.parent();
            }
        }
    }
}

```

```

        None
    }
    Combinator::AdjacentSibling => {
        let right_node = get_node(tree, right_id)?;
        let parent_id = right_node.parent()?;
        let parent_node = get_node(tree, parent_id)?;
        let siblings = parent_node.children();
        let right_pos = siblings.iter().position(|id| *id ==
right_id)?;

        if right_pos == 0 {
            return None;
        }

        let candidate_id = siblings[right_pos - 1];
        let candidate_node = get_node(tree, candidate_id)?;
        if matches_simple_selector(candidate_node, left_selector) {
            Some(candidate_id)
        } else {
            None
        }
    }
    Combinator::GeneralSibling => {
        let right_node = get_node(tree, right_id)?;
        let parent_id = right_node.parent()?;
        let parent_node = get_node(tree, parent_id)?;
        let siblings = parent_node.children();
        let right_pos = siblings.iter().position(|id| *id ==
right_id)?;

        for candidate_id in siblings[..right_pos].iter().rev() {
            let candidate_node = get_node(tree, *candidate_id)?;
            if matches_simple_selector(candidate_node,
left_selector) {
                return Some(*candidate_id);
            }
        }

        None
    }
}

```

Implementasi CSS selector pada aplikasi ini dibangun di atas dua enum utama. Enum Combinator merepresentasikan empat jenis hubungan antar-selector, yaitu Child, Descendant, AdjacentSibling, dan GeneralSibling. Sementara itu, enum SimpleSelector merepresentasikan jenis selector dasar yang dapat dievaluasi pada satu node, mencakup Universal (memilih semua elemen), Tag (berdasarkan nama tag), Class (berdasarkan atribut class), Id (berdasarkan atribut id), dan AttrEq (berdasarkan pasangan key-value atribut tertentu). Setiap bagian dari selector yang telah diparsing direpresentasikan sebagai SelectorPart, yang menyimpan sebuah SimpleSelector beserta informasi combinator yang menghubungkannya ke bagian selector di sebelah kirinya. Kumpulan dari SelectorPart ini disimpan dalam struct SelectorQuery dalam urutan kanan ke kiri, sehingga bagian paling kanan dari selector yang langsung dicocokkan dengan node target selalu berada di indeks pertama.

Proses parsing selector dilakukan oleh fungsi `parse_selector` yang memproses string selector karakter per karakter. Karakter `>`, `+`, dan `~` langsung dikenali sebagai combinator eksplisit, sedangkan karakter spasi diartikan sebagai potensi Descendant Combinator apabila sebelumnya sudah terdapat token selector yang valid. Satu hal yang perlu diperhatikan adalah penanganan tanda kurung siku `[` dan `]` untuk selector atribut seperti `[class=container]`. Selama parser berada di dalam kurung siku yang ditandai dengan variabel `bracket_depth` lebih dari nol, karakter spasi dan combinator tidak diproses sebagai pemisah melainkan diperlakukan sebagai bagian dari token atribut. Setelah seluruh karakter diproses, token-token selector dan combinator yang terkumpul divalidasi dengan memastikan jumlah combinator tepat satu kurang dari jumlah selector, kemudian digabungkan menjadi daftar SelectorPart yang disusun dari kanan ke kiri.

Pencocokan node dengan selector dilakukan oleh fungsi `query_matches_node`. Fungsi ini menerima sebuah node sebagai kandidat dan memeriksa apakah node tersebut cocok dengan keseluruhan selector secara iteratif dari kanan ke kiri. Pada setiap iterasi, node yang sedang diperiksa dicocokkan dengan SimpleSelector pada bagian tersebut melalui fungsi `matches_simple_selector`. Jika tidak cocok, fungsi langsung mengembalikan false. Apabila cocok dan masih terdapat combinator yang perlu dievaluasi, fungsi `match_left_node` dipanggil untuk menemukan node berikutnya yang harus dicocokkan dengan bagian selector di sebelah kiri. Node hasil temuan tersebut kemudian menjadi node aktif pada iterasi berikutnya hingga seluruh bagian selector berhasil dicocokkan.

Logika pencarian node konteks berdasarkan combinator ditangani sepenuhnya oleh fungsi `match_left_node`. Untuk Child Combinator, fungsi mengambil parent langsung dari node kanan dan memeriksa apakah parent tersebut cocok dengan selector kiri, dan jika tidak cocok langsung mengembalikan None tanpa alternatif lain. Untuk Descendant Combinator, fungsi

menelusuri ke atas dari parent node kanan hingga root sambil memeriksa setiap ancestor satu per satu, di mana fungsi `is_ancestor_id` dari indeks LCA digunakan untuk memastikan bahwa node yang diperiksa memang benar-benar ancestor dari node kanan. Untuk Adjacent Sibling Combinator, fungsi mengambil daftar children dari parent node kanan, mencari posisi node kanan dalam daftar tersebut, lalu memeriksa node yang berada tepat satu posisi sebelumnya. Untuk General Sibling Combinator, fungsi melakukan hal serupa namun menelusuri seluruh sibling di sebelah kiri node kanan dari yang paling dekat hingga yang paling jauh, dan sibling pertama yang cocok akan segera dikembalikan sebagai hasilnya.

4.2. Pengujian

4.2.1. Selektor Dasar

A. Tag Selector

Tag <p>	
	BFS
Tag <div>	
DFS	BFS

masukkan URL, selector CSS, dan pilih algoritma BFS atau DFS

URL Halaman HTML

https://www.w3schools.com/css/css_selector

URL akan diparsing menjadi DOM tree

ExampleDOMView

CSS Selector

div

Metode Pencarian

Depth-First Search (DFS)

Max Depth Visualizer (optional)

3

Jika di klik, visualisasi akan pada depth tertentu dan akan menampilkan hirarki paling minimal CSS Selector match dengan node.

Cari Elemen

JALUR AKHIR TRAVERSAL

document -> html -> body -> div

HASIL TRAVERSAL

Waktu Eksekusi: 2024 µs

Node Ditemukan: 585

Visualisasi DOM tree

RIBAYAT REQUEST

https://www.w3schools.com/css/css_selectors.asp

Selector: div (Metode DFS) (Match: 585) (20.41 µs)

https://www.w3schools.com/css/css_selectors.asp

20.41

algoritma BFS atau DFS

URL Halaman HTML

https://www.w3schools.com/css/css_selector

URL akan diparsing menjadi DOM tree

ExampleDOMView

CSS Selector

div

Metode Pencarian

Depth-First Search (DFS)

Max Depth Visualizer (optional)

3

Jika di klik, visualisasi akan pada depth tertentu dan akan menampilkan hirarki paling minimal CSS Selector match dengan node.

Cari Elemen

JALUR AKHIR TRAVERSAL

document -> html -> body -> div

HASIL TRAVERSAL

Waktu Eksekusi: 1249 µs

Node Ditemukan: 585

Visualisasi DOM tree

RIBAYAT REQUEST

https://www.w3schools.com/css/css_selectors.asp

Selector: div (Metode DFS) (Match: 585) (1249 µs)

https://www.w3schools.com/css/css_selectors.asp

23.51

masukkan URL, selector CSS, dan pilih algoritma BFS atau DFS

URL Halaman HTML

https://www.w3schools.com/css/css_selector

URL akan diparsing menjadi DOM tree

ExampleDOMView

CSS Selector

div

Metode Pencarian

Breadth-First Search (BFS)

Max Depth Visualizer (optional)

3

Jika di klik, visualisasi akan pada depth tertentu dan akan menampilkan hirarki paling minimal CSS Selector match dengan node.

Cari Elemen

JALUR AKHIR TRAVERSAL

document -> html -> body -> div

HASIL TRAVERSAL

Waktu Eksekusi: 3078 µs

Node Ditemukan: 585

Visualisasi DOM tree

RIBAYAT REQUEST

https://www.w3schools.com/css/css_selectors.asp

Selector: div (Metode BFS) (Match: 585) (3078 µs)

https://www.w3schools.com/css/css_selectors.asp

23.43

algoritma BFS atau DFS

URL Halaman HTML

https://www.w3schools.com/css/css_selector

URL akan diparsing menjadi DOM tree

ExampleDOMView

CSS Selector

div

Metode Pencarian

Breadth-First Search (BFS)

Max Depth Visualizer (optional)

3

Jika di klik, visualisasi akan pada depth tertentu dan akan menampilkan hirarki paling minimal CSS Selector match dengan node.

Cari Elemen

JALUR AKHIR TRAVERSAL

document -> html -> body -> div

HASIL TRAVERSAL

Waktu Eksekusi: 2333 µs

Node Ditemukan: 585

Visualisasi DOM tree

RIBAYAT REQUEST

https://www.w3schools.com/css/css_selectors.asp

Selector: div (Metode BFS) (Match: 585) (2333 µs)

https://www.w3schools.com/css/css_selectors.asp

23.50

Tag <a>

DFS

BFS

masukkan URL, selector CSS, dan pilih algoritma BFS atau DFS

URL Halaman HTML

https://www.w3schools.com/css/css_selector

URL akan diparsing menjadi DOM tree

ExampleDOMView

CSS Selector

a

Metode Pencarian

Depth-First Search (DFS)

Max Depth Visualizer (optional)

3

Jika di klik, visualisasi akan pada depth tertentu dan akan menampilkan hirarki paling minimal CSS Selector match dengan node.

Cari Elemen

JALUR AKHIR TRAVERSAL

document -> html -> body -> div -> a -> a

HASIL TRAVERSAL

Waktu Eksekusi: 2017 µs

Node Ditemukan: 1152

Visualisasi DOM tree

RIBAYAT REQUEST

https://www.w3schools.com/css/css_selectors.asp

Selector: a (Metode DFS) (Match: 1152) (20.41 µs)

https://www.w3schools.com/css/css_selectors.asp

23.43

masukkan URL, selector CSS, dan pilih algoritma BFS atau DFS

URL Halaman HTML

https://www.w3schools.com/css/css_selector

URL akan diparsing menjadi DOM tree

ExampleDOMView

CSS Selector

a

Metode Pencarian

Breadth-First Search (BFS)

Max Depth Visualizer (optional)

3

Jika di klik, visualisasi akan pada depth tertentu dan akan menampilkan hirarki paling minimal CSS Selector match dengan node.

Cari Elemen

JALUR AKHIR TRAVERSAL

document -> html -> body -> div -> a -> a

HASIL TRAVERSAL

Waktu Eksekusi: 2268 µs

Node Ditemukan: 1152

Visualisasi DOM tree

RIBAYAT REQUEST

https://www.w3schools.com/css/css_selectors.asp

Selector: a (Metode BFS) (Match: 1152) (2268 µs)

https://www.w3schools.com/css/css_selectors.asp

23.44

Object Model Traversal Explorer

masukkan URL, halaman, selector CSS, lalu pilih algoritma BFS atau DFS

URL Halaman HTML

https://www.w3schools.com/css/css_selector

URL akan diparsing menjadi DOM tree

ExampleDOMView

CSS Selector

a

Metode Pencarian

Depth-First Search (DFS)

Max Depth Visualizer (optional)

3

Jika di klik, visualisasi akan pada depth tertentu dan akan menampilkan hirarki paling minimal CSS Selector match dengan node.

Cari Elemen

JALUR AKHIR TRAVERSAL

document -> html -> body -> div -> a -> a

HASIL TRAVERSAL

Waktu Eksekusi: 1916 µs

Visualisasi DOM tree

RIBAYAT REQUEST

https://www.w3schools.com/css/css_selectors.asp

23.51

masukkan URL, selector CSS, dan pilih algoritma BFS atau DFS

URL Halaman HTML

https://www.w3schools.com/css/css_selector

URL akan diparsing menjadi DOM tree

ExampleDOMView

CSS Selector

a

Metode Pencarian

Breadth-First Search (BFS)

Max Depth Visualizer (optional)

3

Jika di klik, visualisasi akan pada depth tertentu dan akan menampilkan hirarki paling minimal CSS Selector match dengan node.

Cari Elemen

JALUR AKHIR TRAVERSAL

document -> html -> body -> div -> a -> a

HASIL TRAVERSAL

Waktu Eksekusi: 2513 µs

Node Ditemukan: 1152

Visualisasi DOM tree

RIBAYAT REQUEST

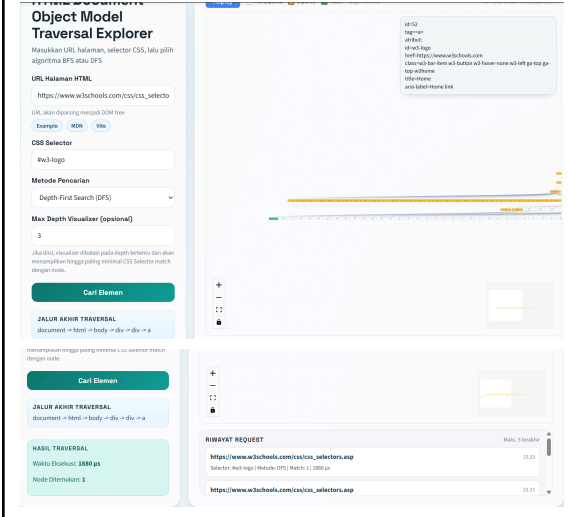
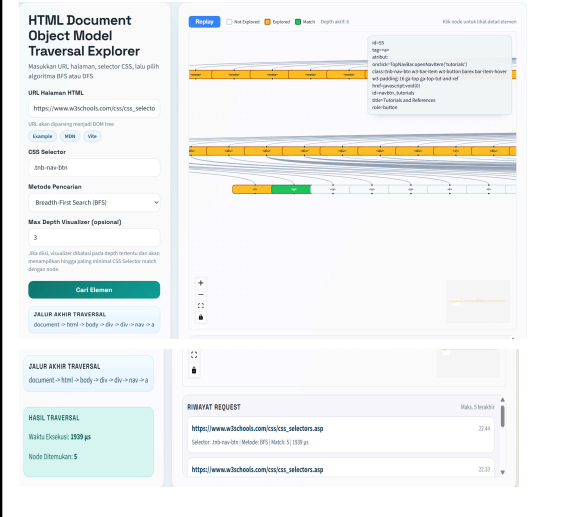
https://www.w3schools.com/css/css_selectors.asp

Selector: a (Metode BFS) (Match: 1152) (2513 µs)

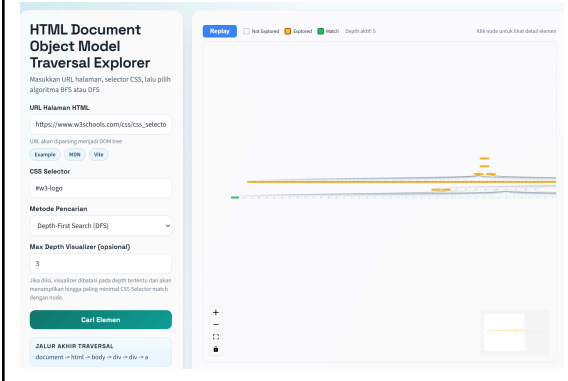
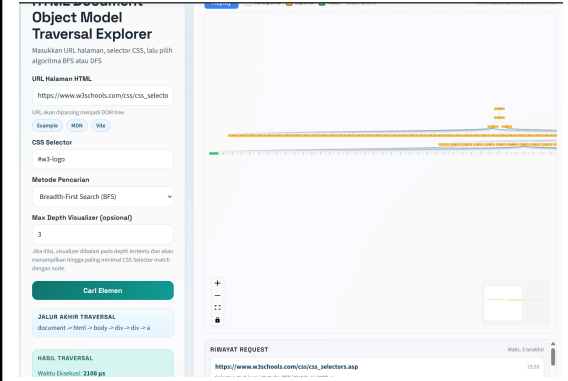
https://www.w3schools.com/css/css_selectors.asp

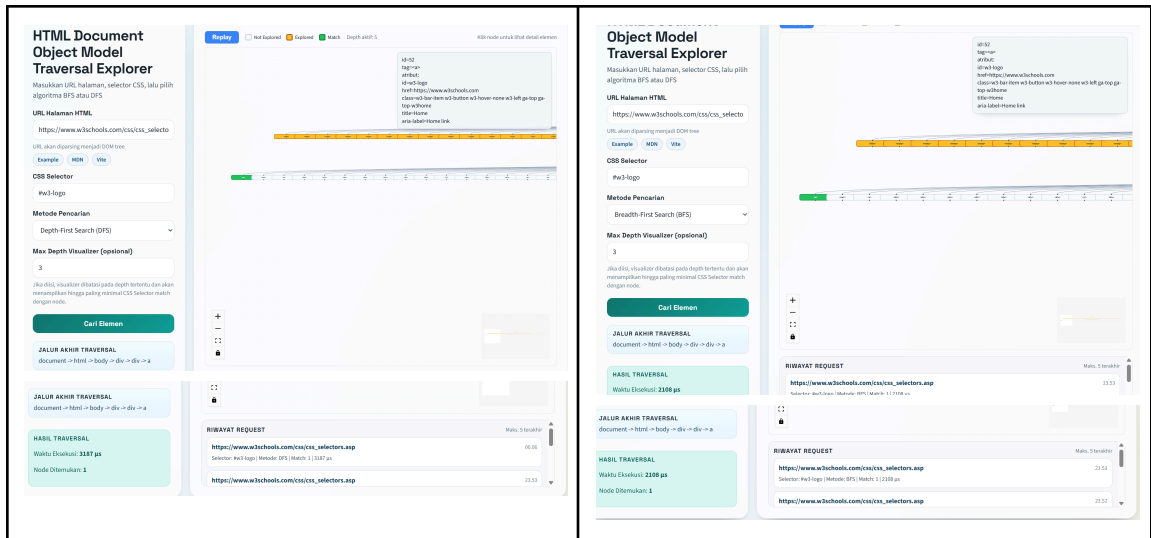
23.50

B. Class Selector

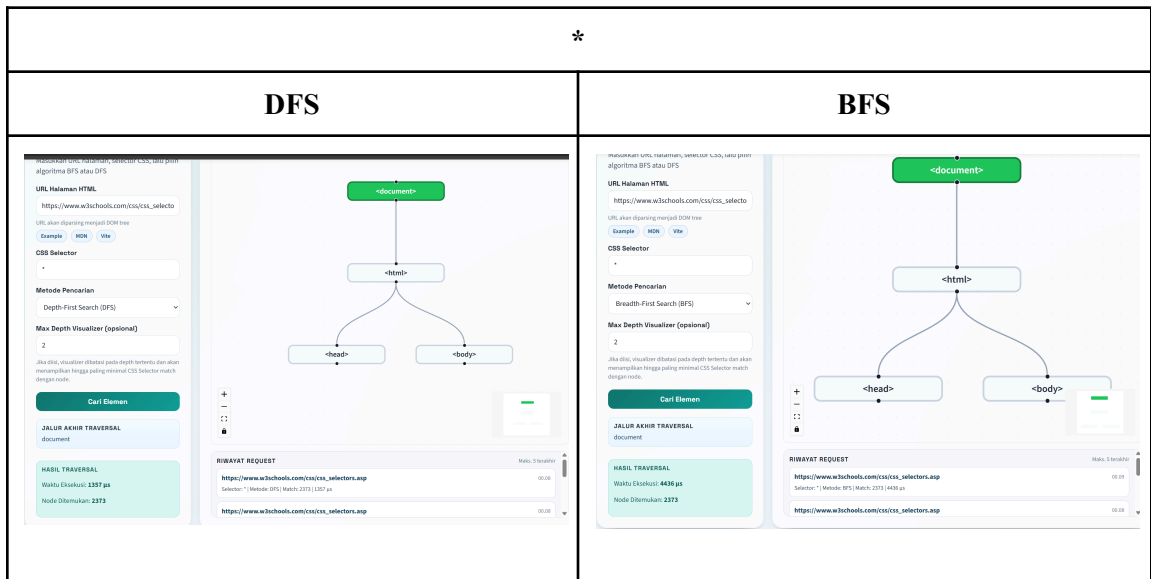
.tnb-nav-btn	
DFS	BFS
	

C. ID Selector

#w3-logo	
DFS	BFS
	



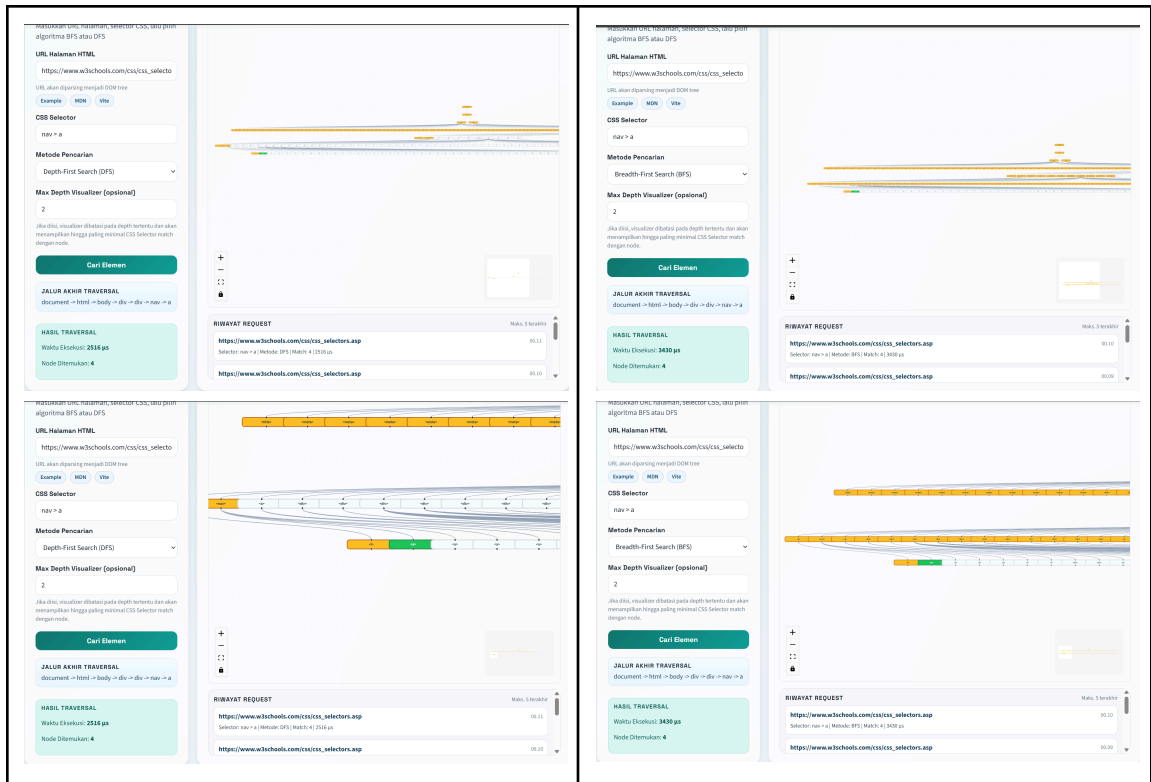
D. Universal Selector



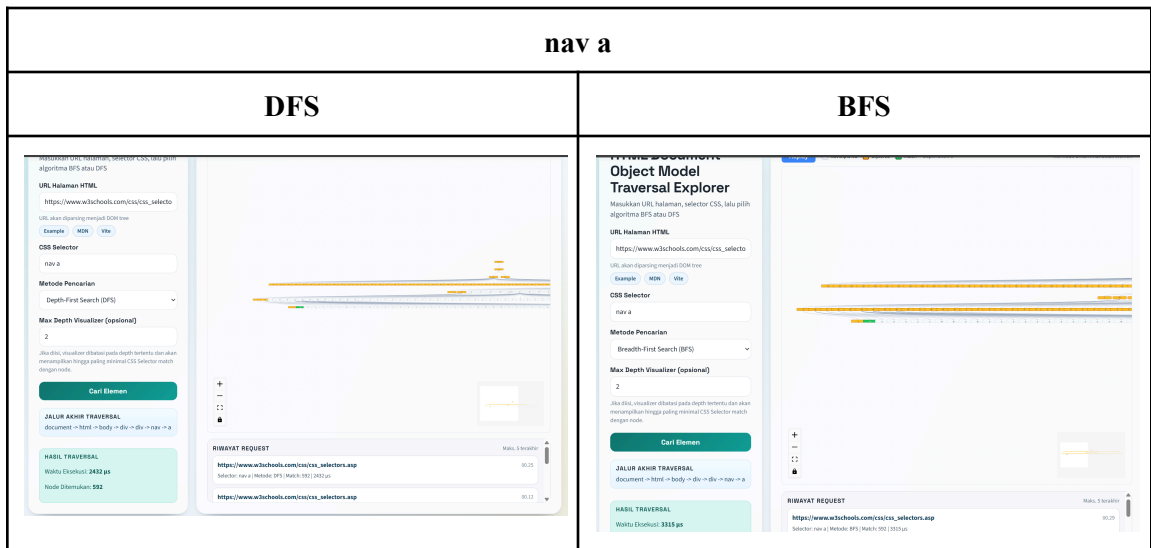
4.2.2 Selektor Combinator

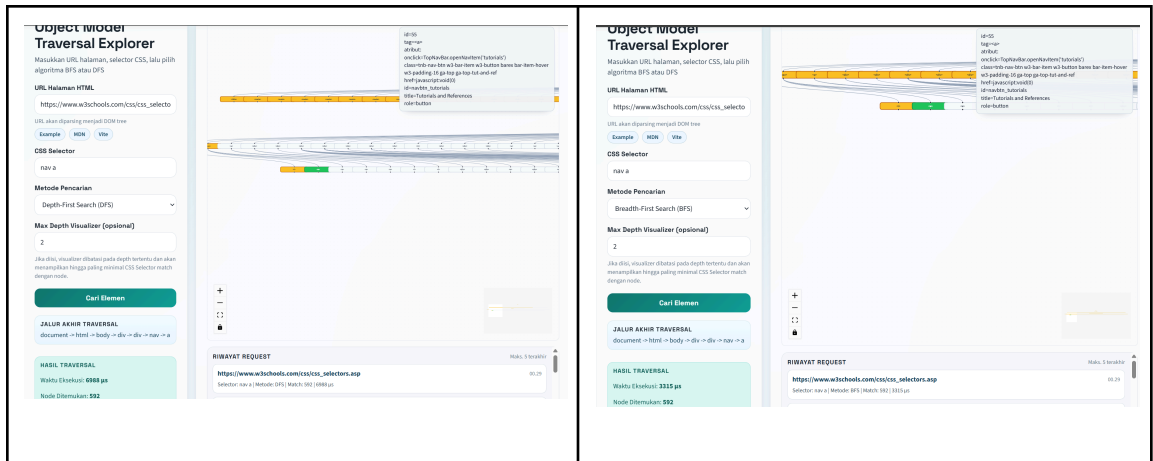
A. Child Combinator (>)

nav > a	
DFS	BFS



B. Descendent Combinator (“ “)

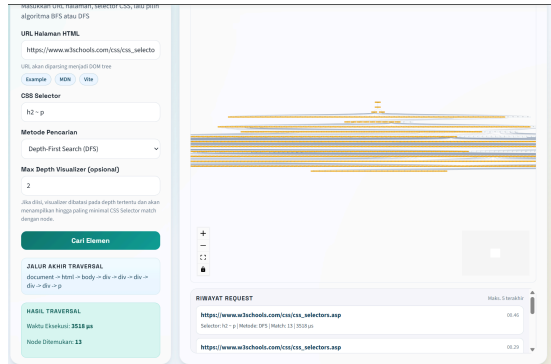
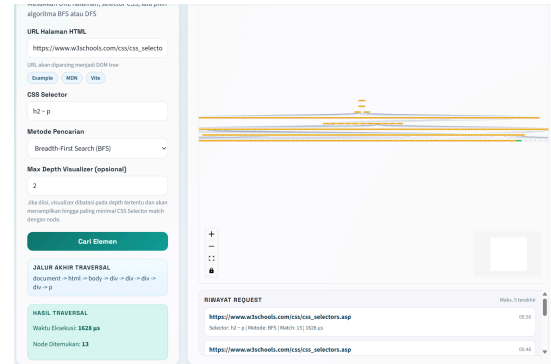
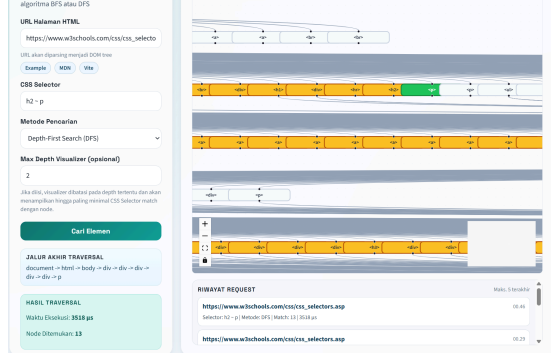
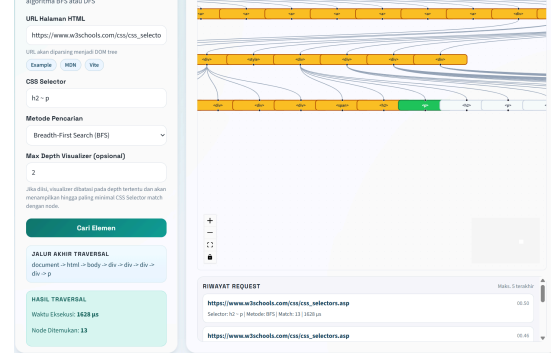




C. Adjacent Sibling Combinator (+)

h2 + p	
DFS	BFS
Masukkan URL halaman, selector CSS, lalu pilih algoritma BFS atau DFS URL Halaman HTML https://www.w3schools.com/css/css_selectors URL akan diparsing menjadi DOM tree Example DOM View CSS Selector h2 + p Metode Pencarian Depth-First Search (DFS) Max Depth Visualizer (optional) 2 Jika diis, visualizer dibatasi pada depth tertentu dan akan menampilkan hingga paling minimal CSS Selector match dengan node. Cari Elemen JALUR AKHIR TRAVERSAL document -> html -> body -> div -> div -> div -> div -> p HASIL TRAVERSAL Waktu Eksekusi: 4375 µs Node Ditemukan: 6 RIBAYAT REQUEST https://www.w3schools.com/css/css_selectors.asp Selector: h2 + p / Metode: DFS / Match: 6 / 4375 µs https://www.w3schools.com/css/css_selectors.asp	Masukkan URL halaman, selector CSS, lalu pilih algoritma BFS atau DFS URL Halaman HTML https://www.w3schools.com/css/css_selectors URL akan diparsing menjadi DOM tree Example DOM View CSS Selector h2 + p Metode Pencarian Breadth-First Search (BFS) Max Depth Visualizer (optional) 2 Jika diis, visualizer dibatasi pada depth tertentu dan akan menampilkan hingga paling minimal CSS Selector match dengan node. Cari Elemen JALUR AKHIR TRAVERSAL document -> html -> body -> div -> div -> div -> div -> p HASIL TRAVERSAL Waktu Eksekusi: 3938 µs Node Ditemukan: 6 RIBAYAT REQUEST https://www.w3schools.com/css/css_selectors.asp Selector: h2 + p / Metode: BFS / Match: 6 / 3938 µs https://www.w3schools.com/css/css_selectors.asp
Masukkan URL halaman, selector CSS, lalu pilih algoritma BFS atau DFS URL Halaman HTML https://www.w3schools.com/css/css_selectors URL akan diparsing menjadi DOM tree Example DOM View CSS Selector h2 + p Metode Pencarian Depth-First Search (DFS) Max Depth Visualizer (optional) 2 Jika diis, visualizer dibatasi pada depth tertentu dan akan menampilkan hingga paling minimal CSS Selector match dengan node. Cari Elemen JALUR AKHIR TRAVERSAL document -> html -> body -> div -> div -> div -> div -> p HASIL TRAVERSAL Waktu Eksekusi: 4375 µs Node Ditemukan: 6 RIBAYAT REQUEST https://www.w3schools.com/css/css_selectors.asp Selector: h2 + p / Metode: DFS / Match: 6 / 4375 µs https://www.w3schools.com/css/css_selectors.asp	Masukkan URL halaman, selector CSS, lalu pilih algoritma BFS atau DFS URL Halaman HTML https://www.w3schools.com/css/css_selectors URL akan diparsing menjadi DOM tree Example DOM View CSS Selector h2 + p Metode Pencarian Breadth-First Search (BFS) Max Depth Visualizer (optional) 2 Jika diis, visualizer dibatasi pada depth tertentu dan akan menampilkan hingga paling minimal CSS Selector match dengan node. Cari Elemen JALUR AKHIR TRAVERSAL document -> html -> body -> div -> div -> div -> div -> p HASIL TRAVERSAL Waktu Eksekusi: 3938 µs Node Ditemukan: 6 RIBAYAT REQUEST https://www.w3schools.com/css/css_selectors.asp Selector: h2 + p / Metode: BFS / Match: 6 / 3938 µs https://www.w3schools.com/css/css_selectors.asp

D. General Sibling Combinator (~)

h2 ~ p	
DFS	BFS
	
	

Bab 5

Kesimpulan dan Saran

5.1. Kesimpulan

Aplikasi CSS Selector pada pohon DOM berhasil dibangun dengan mengimplementasikan algoritma BFS dan DFS secara penuh, termasuk varian concurrent yang memanfaatkan multithreading. Kedua algoritma terbukti dapat digunakan untuk menemukan elemen HTML berdasarkan CSS selector yang diberikan, namun dengan karakteristik yang berbeda. BFS lebih unggul ketika elemen target berada di level atas pohon karena sifatnya yang menjelajahi pohon secara melebar, sementara DFS lebih efisien ketika elemen target tersembunyi jauh di dalam salah satu cabang pohon.

Penerapan LCA Binary Lifting terbukti memperkuat kemampuan evaluasi CSS selector, khususnya untuk Descendant Combinator dan General Sibling Combinator yang memerlukan pemeriksaan hubungan hirarkis antar-node. Dengan pra komputasi tabel binary lifting, pengecekan relasi ancestor yang semula membutuhkan penelusuran linear dari node ke root dapat dilakukan dengan kompleksitas $O(\log n)$, sehingga evaluasi selector menjadi lebih efisien pada pohon DOM yang besar.

Secara keseluruhan, proyek ini memperlihatkan bahwa struktur DOM sebagai pohon sangat cocok untuk dieksplorasi menggunakan algoritma traversal graf, dan bahwa pemilihan algoritma yang tepat bergantung pada karakteristik dokumen HTML yang ditelusuri serta posisi perkiraan elemen target di dalam pohon.

5.2. Saran

Untuk pengembangan lebih lanjut, cakupan CSS selector yang didukung dapat diperluas, misalnya dengan menambahkan pseudo-class seperti `:first-child`, `:nth-child`, atau `:not()`, yang saat ini belum diimplementasikan. Selain itu, parser HTML yang digunakan masih bersifat sederhana dan belum sepenuhnya toleran terhadap seluruh bentuk markup yang tidak valid di dunia nyata, sehingga ketahanan parser dapat ditingkatkan untuk menangani lebih banyak kasus tepi. Dari sisi performa, strategi multithreading yang digunakan saat ini masih terbatas pada pembagian per level untuk BFS dan per subtree root untuk DFS, sehingga masih ada ruang untuk eksplorasi strategi paralelisasi yang lebih adaptif terhadap bentuk pohon yang beragam.

Lampiran

Tautan repository GitHub: https://github.com/LeonArif/Tubes2_RustDz.git

Tabel 6.1 Poin Pengerjaan Tugas

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video		✓
10	[Bonus] Deploy aplikasi		✓
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading	✓	
13	[Bonus] Implementasi LCA Binary Lifting	✓	

Tabel 6.2 Pembagian Tugas

Nama	NIM	Pembagian Tugas
Stanislaus Ardy Bramantyo	18223057	Membuat Animasi dan Visualisasi Tree, Mengimplementasikan algoritma DFS, BFS, Menulis Laporan
Leonard Arif Sutiono	18223120	Membuat Backend, Mengimplementasikan algoritma DFS, BFS. Mengimplementasikan LCA Binary Lifting, Membuat Docker, Menulis Laporan

Muhammad Dzaky Atha Fadhilah	18223124	Mengintegrasikan Backend, Membuat Design Frontend, Mengimplementasikan algoritma DFS, BFS, Menulis Laporan
------------------------------	----------	--

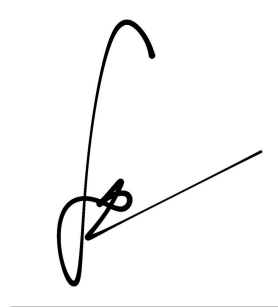
Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Stanislaus Ardy Bramantyo



Muhammad Dzaky A.F.



Leonard Arif Sutiono

Daftar Pustaka

Algoritma BFS dan DFS

[https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/13-BFS-DFS-\(2026\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/13-BFS-DFS-(2026)-Bagian1.pdf)

[https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/14-BFS-DFS-\(2026\)-Bagian2.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/14-BFS-DFS-(2026)-Bagian2.pdf)

<https://graphable.ai/blog/best-graph-traversal-algorithms/#:~:text=As%20mentioned%20in%20the%20introduction,depth%20level%20in%20the%20graph.>

<https://brilliant.org/wiki/depth-first-search-dfs/>

[https://brilliant.org/wiki/breadth-first-search-bfs/#:~:text=Breadth%2Dfirst%20search%20\(BFS\)%20is%20an%20important%20graph%20search,of%20in%20terms%20of%20graphs.](https://brilliant.org/wiki/breadth-first-search-bfs/#:~:text=Breadth%2Dfirst%20search%20(BFS)%20is%20an%20important%20graph%20search,of%20in%20terms%20of%20graphs.)

Document Object Model

<https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/>

<https://tutorial.techaltum.com/htmlTags.html>

https://www.reddit.com/r/webdev/comments/ur6v5m/css_selectors_visually_explained/

<https://stackoverflow.com/questions/3150293/how-does-a-parser-for-example-html-work>

<https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Selectors>

Client-Server Architecture

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/First_steps/Client-Server_overview

Rust

<https://doc.rust-lang.org/stable/>

<https://youtu.be/FDWKlJmHv6k?si=l5gnboFaKMVBEAC8>